

Composable cancellation for sender-based async operations

Document #: P2175R0
Date: 2020-12-14
Project: Programming Language C++
Audience: SG1 - Concurrency Sub-Group
Reply-to: Lewis Baker
<lewissbaker@gmail.com>

Contents

- 1 Abstract
- 2 Motivation
 - 2.1 Why Cancellation?
 - 2.2 Structured Concurrency
- 3 Prior-art in cancellation patterns
 - 3.1 Networking TS / boost::asio cancellation model
 - 3.2 .NET Framework cancellation model
 - 3.3 cppcoro::task cancellation
 - 3.4 folly::coro cancellation model
 - 3.5 std::stop_token
- 4 Proposed Changes
 - 4.1 Add stoppable_token and related concepts
 - 4.2 Tweaks to std::stop_token
 - 4.3 Add std::never_stop_token
 - 4.4 Add std::in_place_stop_token, std::in_place_stop_source, std::in_place_stop_callback<CB>
 - 4.5 Add get_stop_token() customisation-point
 - 4.6 Type-traits
- 5 Design Discussion
 - 5.1 Naming
 - 5.2 Why do we need a std::stoppable_token concept?

- 5.3 Should `unstoppable_token` concept just be a trait?
- 5.4 Can `stoppable_token_for` concept be recast as semantic requirements?
- 5.5 Why do we need the `::callback_type<CB>` type-alias on the stop-token type?
- 5.6 Why doesn't paper this propose adding a `std::never_stop_callback<CB>` type-name?
- 5.7 Should `get_stop_token()` be applicable to more than receivers?
- 5.8 Composability
- 5.9 Cancellation is optional / best-effort
- 5.10 Performance Considerations
- 5.11 Limitations of `sender_traits::sends_done`
- 5.12 Cancellation support for scheduler implementations
- 5.13 Impacts to other proposals
- 5.14 Future work
- 6 Implementation Experience
- 7 Wording
- 8 Appendices
 - 8.1 Appendix A: The `stop_when()` algorithm

§ 1 Abstract

This paper proposes a general-purpose, composable mechanism for requesting cancellation of in-flight sender-based async operations as an extension to [P0443R14](#) - “A Unified Executors Proposal for C++.”

The paper [P0443R14](#) proposes adding the sender/receiver concepts which provide a generic interface for asynchronous operations. As part of this interface, when an async operation completes it completes with one of three possible kinds of signals, delivered by invoking one of the `set_value`, `set_error` or `set_done` customisation-points on the receiver.

The first two correspond roughly to the async-equivalent of an ordinary function completing with a return-value or an exception, respectively, and the `set_done` completion-signal is a third kind of completion that represents a result that was neither success nor failure - typically used to signal that the operation completed early because it was asked to do so by the caller.

While [P0443R14](#) provides the ability for an async operation to complete with an empty result in the case it was cancelled, **there is currently no standard mechanism specified**

to allow the caller to communicate this request to stop an async operation once it has been started.

This leaves individual async operations having to use ad-hoc mechanisms to allow the caller to communicate a request to cancel. Examples include passing a `std::chrono::duration` as a timeout parameter for time-based cancellation, passing a `std::stop_token` as a parameter to the async function/coroutine, or adding a `.cancel()` method on the same object as the async method (as with the Networking TS).

For cancellation to compose well within an application, it generally needs to be supported at all levels. For example, for a request to cancel a high-level operation to be effective, that request to cancel an operation needs to be able to be communicated through each layer of the application to the leaf-level operations. e.g. to a timer, I/O operation or compute loop on which the high-level operation's completion is dependent.

The use of ad-hoc mechanisms, however, makes it difficult to compose and propagate cancellation through intermediate layers. This is especially true for generic algorithms that compose already-constructed senders, such as the `when_all()` algorithm, and thus have no way to control the parameters passed to the async operation that created the sender.

Defining a standard mechanism for communicating a request for cancellation of async operations is essential to allow building generic async algorithms that support cancellation.

The high-level goals of the cancellation design proposed by this paper are:

- Define a standard cancellation mechanism that is generically composable. This allows algorithms to be built that can either be transparent to cancellation or that can introduce new cancellation scopes and inject cancellation requests into child operations. e.g. allow defining a generic `timeout()` algorithm that can request cancellation of a child operation after a given time has elapsed.
- Allow overhead of cancellation mechanisms to compile-out to nothing when we can statically determine that cancellation will never be requested. i.e. “Don’t pay for what you don’t use”
- Don’t force callers or callees to have to support cancellation.
 - If an async operation doesn’t support cancellation then it doesn’t have to write any code to opt-out of cancellation. Supporting cancellation is opt-in.
 - If a caller will never request cancellation then they shouldn’t have to do any work to opt-out of support for cancellation.

This paper proposes the following changes to define a standard cancellation mechanism for sender-based operations:

- Add a named concept, `std::stoppable_token`, that matches types with the same shape as `std::stop_token`. This allows alternative implementations of the

stop_token synchronisation primitive that can be more efficient for certain use-cases.

- Add two additional concepts which are refinements of `std::stoppable_token`:
 - `std::stoppable_token_for<CB, Initializer>` - tests that, in addition to satisfying the `std::stoppable_token` concept that you can also construct a `T::callback_type<CB>` from an instance of the stop-token and a value of type `Initializer`.
 - `std::unstoppable_token` - A `std::stoppable_token` for which `stop_possible()` is static-constexpr and always returns false.
- Add `get_stop_token()` customisation-point, invocable on the receiver passed to `connect()` to obtain the stop-token to use for that operation.
- Add new type-traits for determining the stop-token type associated with a receiver:
 - `std::stop_token_type_t<T>` obtains the decayed result-type of `get_stop_token()` invoked on an argument of type `T`.
- Add two new types that satisfy the `stoppable_token` concept:
 - `std::never_stop_token` - used for cases where cancellation is not required
 - `std::in_place_stop_token` - used for cases where the stop-source does not need to be movable/copyable and the lifetime of uses of the stop-token are strictly nested within the lifetime of the corresponding stop-source.
- Update `std::stop_token` to add the nested `::callback_type` type-alias required by the `stoppable_token` concept.

This also has impacts on other proposals:

- Impacts on sender-based algorithms in [P1897R3](#) which need to be updated to support cancellation.
- Builds on top of `tag_invoke()` mechanism proposed in [P1895R0](#) for customisation-points to generalise the mechanism for passing context from caller to callee through CPOs on the receiver.
- Impacts on `std::task/lazy` proposed in [P1056R1](#) to allow cancellation requests to propagate through coroutines to awaited senders.
- Potential impacts on interaction with cancellation in the Networking TS.

The facilities proposed in this library have been implemented in the [libunifex](#) opensource library.

§ 2 Motivation

This section will attempt to explain why the author believes cancellation to be a fundamental building block of concurrent programs, and why it's necessary to support writing concurrent code in a structured way.

§ 2.1 Why Cancellation?

§ 2.1.1 Ordinary functions

When we write sequential, single-threaded synchronous code in C++ we call a function when we want it to produce some result for us.

The caller presumably needs the result of that function to achieve its goal, otherwise it wouldn't have called the function. You can think of “goal” here as roughly equivalent to “satisfying its post-conditions.”

If the function call completes with an error (i.e. an exception) then this generally indicates that the callee was unable to achieve its goal. Thus the caller's current strategy for achieving its goal has failed and so the caller can either handle the error and try a different strategy for achieving its goal, or if the caller does not have an alternative strategy for achieving its goal then it can let the error propagate to its caller as a way of saying that it was not able to achieve its goal.

Either way, a function call will either complete with a value, indicating success, and the caller will continue executing along its value continuation-path, executing the next step towards achieving its goal, or it will complete with an error, indicating failure, and the caller will execute along the error continuation-path (typically by unwinding until it hits an exception handler).

As the caller is suspended while the callee is executing and the program is single-threaded, in many cases there is nothing that can change the fact that the caller needs that result in order to achieve its goal.

A sequential, single-threaded program can only pursue one strategy for achieving its goal at a time and so needs to wait until the result of the current strategy is known, at which point it can then make a choice about what to do next.

§ 2.1.2 Concurrency introduces a need for cancellation

Once we allow a program to execute multiple operations concurrently, it is possible that we might have multiple strategies for achieving the goal of a program that are executing at the same time.

For example, the goal of (a part of) the program might be “try to download this file or stop trying after 30s,” or it might be “try to connect to each of these 3 servers and send a request to whichever one connects first.”

If we take the file-downloading example, this program might concurrently try download the file and also start waiting until 30s has elapsed. If one of these strategies is successful (e.g. we finished waiting for 30s) then this means that the other strategy (i.e. downloading the file) is no longer needed as we have already satisfied the goal of this part of the program.

This now means that we have an operation that is currently executing whose result is no longer required. It would be a waste of resources to let this operation continue trying to fulfil its goal - it may still be consuming CPU-cycles, memory or network bandwidth.

Ideally, to avoid unnecessarily wasting these resources, we want this operation to stop executing as soon as possible so that it can free the resources it is using. This means we need a way to send a request to this operation to tell it to stop executing.

Note that concurrency here doesn't necessarily require multiple threads in the current process. It might involve some kind of entity external to the process that is making progress independently and that communicates with this process somehow. e.g. via I/O.

For example, another process on the same host that communicates via a pipe, a process on another host that communicates over the network, the processor running on an attached storage device, or even a clock chip that keeps track of the current time.

§ 2.1.3 Concurrency Algorithm Examples

To give some more context, let's look at a few concurrency-related algorithms that involve a need for cancellation:

Example 1: `when_all(senders...)`

The first example is the `when_all()` algorithm, proposed in [P1897R3](#).

This algorithm takes a variadic number of input senders and returns a sender that launches each of those senders, allowing them to potentially execute concurrently. The operation completes when all of the input senders have completed, producing a pack of variants of tuples containing the value-result of each of the corresponding input senders.

However, if any of the input senders completes with either `set_error` or `set_done` then the whole operation completes with the first such result. Therefore, completing with `set_error` or `set_done` means that we have nowhere for the results of other operations to go; the results of other operations are just discarded.

In this case, it would be beneficial if we could request that any of the outstanding operations whose results will be discarded stop as soon as possible so that the overall operation can complete more quickly and avoid using more resources than it needs to. e.g. cpu, memory, network-bandwidth, etc.

Example 2: `stop_when(source, trigger)`

The second example is the `stop_when()` algorithm, which is available in the `libunifex` library.

This algorithm takes as input two senders and returns a new sender that launches each of those senders, allowing them to potentially execute concurrently.

When one of the senders completes it requests cancellation of the other sender.

The whole operation completes with the result of source and the result of trigger is discarded. The trigger result is used purely as a trigger to request cancellation of source.

This algorithm can be used whenever a particular operation should be cancelled when some event occurs and that event is represented as a sender. For example, it could be an event that fires at a particular time, using `schedule_at()` on a scheduler.

Example:

```
task example(scheduler auto s) {
    auto dueTime = now(s) + 10s;
    co_await stop_when(some_operation(), schedule_at(s, dueTime));
}
```

Example 3: `timeout(source, duration)`

The `timeout()` algorithm is similar to `stop_when()` in that it requests that the source operation stop when some event triggers. However, it is different in that it is limited to time-based triggering of the cancellation and also, in the case that the timeout duration elapses, overrides the result of source and instead completes with a `timeout_error`.

You can apply the `timeout()` algorithm to any sender that supports cancellation and this will cause the operation to timeout after a given time without that operation having to have explicitly added support for timeouts.

The example from the “Concurrency introduces a need for cancellation” above might be written as:

```
sender auto download_file(url sourceUrl, path destinationPath);

task example() {
    co_await timeout(download_file("https://example.com/file",
        "./file.txt"), 30s);
}
```

Example 4: `first_successful(rangeOfSenders)`

Another variation of a concurrency algorithm is the `first_successful()` algorithm, which takes a collection of input senders and returns a sender that starts each of the input senders, allowing them to execute concurrently.

If any of the operations complete successfully then it completes with the result of the first successful operation. Otherwise, if none of them complete successfully then completes with the result of the last sender to complete.

This algorithm can be used, for example, to attempt to connect to several candidate servers/end-points/transport concurrently and to use whichever connection was established first. This is often referred to as the “happy eyeballs” algorithm.

As soon as a successful result is obtained, the results of any other senders will be discarded. So ideally, the algorithm would request cancellation of any outstanding operations so that we can avoid wasting resources.

Example 5: Composed concurrency/cancellation

Note that with the above algorithms each of them introduce some form of concurrency in the program and have some condition under which they can request cancellation of one or more of their child operations.

However, for these operations to compose, they still need to also be responsive to cancellation requests coming from their parent.

For example, consider:

```
sender auto example() {  
    return timeout(  
        stop_when(  
            when_all(do_a(), do_b(), do_c()),  
            uiPanel.cancel_button_clicked()),  
        30s);  
}
```

In this example, we want to concurrently execute 3 operations; `do_a()`, `do_b()` and `do_c()`, each of which return senders, and then complete once all 3 operations complete.

If we look at the `do_c()` operation, we want to stop this operation early:

- by `when_all()` if one of `do_a()` or `do_b()` fail; or
- by `stop_when()` if the `cancel_button_clicked()` sender completes, triggered by the user clicking a cancel button on the user-interface; or
- by `timeout()` if the operation takes longer than 30s to complete; or
- by the parent/consumer of `example()` if it requests cancellation of the `example()` operation.

A composable cancellation mechanism allows us to encapsulate concurrency and cancellation patterns in generic, reusable algorithms which can be combined in interesting ways without additional ceremony and boilerplate.

§ 2.1.4 Leaf-operation Cancellation Examples

Generally, a program will have some number of potentially-concurrent leaf operations that represent operations that are able to make the next forward step of the program towards its goals.

Non-leaf operations are the other operations, which are unable to make forward progress until one or more of the leaf operations makes forward progress. The algorithms listed in the section above are all non-leaf operations in the program.

At the end of the day, for cancellation to be effective, we need non-leaf operations to be able to forward requests for cancellation through to the leaf operations and we need the leaf operations to be able to respond to requests for cancellation in a timely manner.

Leaf operations that do not respond to cancellation requests should be permitted, however. Some operations may just not be cancellable, while others may have chosen to not implement this support for various reasons.

For context, here are some examples of leaf operations we might want to support cancellation:

Example 1: Cancelling a `schedule()` operation

The paper [P0443R14](#) adds a basis operation named `schedule()` which allows you to schedule work onto the associated execution context of the scheduler.

When the `schedule` operation is started by calling the `start()` customisation-point, it typically enqueues a work-item on to a queue of some sort. A worker thread will eventually dequeue this work-item and when it does, signals completion of the `schedule()` operation by invoking `set_value()` on the receiver. As the operation completes on a worker thread associated with the scheduler, any work done inside the receiver's `set_value()` method will be running on the scheduler's execution context.

The `schedule()` operation here is considered a “leaf” operation and generally needs to be composed with other operations for it to be useful.

e.g. it might be composed with `transform()` to allow calling a function on the associated execution context.

```
execution::transform(  
    execution::schedule(scheduler),  
    [] {  
        // This runs on 'scheduler's execution context.  
        return compute_result();  
    });
```

This can further be composed with an algorithm like `when_all()` to allow multiple operations to be executed concurrently (e.g. on a thread-pool).

```
execution::when_all(  
    execution::transform(  
        execution::schedule(tpScheduler), [] { return compute_a(); } ),  
    execution::transform(  
        execution::schedule(tpScheduler), [] { return compute_b(); } ));
```

If the result of that composed operation is no longer required, e.g. because one of the computations failed with an exception, then ideally we'd be able to cancel the `schedule()` operation for the remaining computation and remove the work-item from the scheduler's queue immediately rather than wait until some worker-thread dequeues it.

If a scheduler is oversubscribed with a large queue depth then being able to cancel a `schedule()` operation that is currently at the back of the queue can reduce the latency of the composed operations.

Example 2: Cancelling a timer

In `libunifex`, schedulers that implement the `time_scheduler` concept provide `schedule_after()` and `schedule_at()` operations in addition to the required `schedule()` operation.

The `schedule_after(scheduler, duration)` operation will complete on the scheduler's execution context with `set_value()` after a delay of `duration` time after the operation is started.

Often time-based scheduling is used for things like timeouts, where the timer will need to be cancelled if the operation completes before the timeout has elapsed. If the timer cannot be cancelled promptly then the timeout operation will have to wait until the timeout duration elapses before it can complete.

Example 3: Cancelling async I/O on Windows

Another common use-case for cancellation is cancelling async I/O.

For example, a user of an application might click on one button, which starts loading a large image from storage, but then immediately clicks on a different button to select a different image. The application should ideally avoid continuing to load the first image if the load had not yet completed to avoid wasting I/O bandwidth which could be better used for loading the second image.

Being able to cancel an async I/O operation that is no longer required can help with application responsiveness.

It can also be used to cause operations to complete that will otherwise never complete. e.g. a network application that is asynchronously attempting to accept a connection may never receive an incoming connection attempt. We may want to stop accepting connections, say during shutdown, and thus cancel the async accept operation which would otherwise be waiting forever.

If we look at the basic async I/O APIs for Windows platforms, an async operation on a file associated with an I/O completion port is started by calling `ReadFile()` or `WriteFile()`, and passing a pointer to an `OVERLAPPED` structure.

The OS signals completion by posting a completion event to the I/O completion port with the corresponding `OVERLAPPED` pointer, which an event-loop typically receives by calling `GetQueuedCompletionStatus()`.

If we want to cancel the I/O operation early then we need to call `CancelIoEx()` and pass the same file-handle and `OVERLAPPED` pointer to issue a request to the OS to cancel the

operation. To be able to do this, we need to be able to subscribe a callback that will be invoked when cancellation of the async read operation is requested so that we can actively call `CancelIoEx()`.

Note that the OS may or may not be able to actually cancel the operation depending on how far along the I/O operation has progressed (there may already be a completion event sitting the I/O completion port's queue) and depending on the capabilities of the I/O device.

§ 2.2 Structured Concurrency

One of the design principles that has been guiding the design of sender/receiver, coroutines and of sender-based concurrency algorithms is that of “structured concurrency.”

The introduction of structured control-flow, which provided composable control-flow structures such as `if/else`, `do/while` and `switch` blocks, made it easier to reason about control-flow compared to code that exclusively used `goto` or conditional branches. These control-flow constructs can be composed arbitrarily to create complex logic and algorithms. Structured control-flow also tied control-flow to program scopes which made it easier to visually see the structure in the control-flow of your program.

Similarly, the introduction of structured-lifetime in C++, which tied lifetime of objects to program scopes through use of destructors, ensured that resources were cleaned up at the end of scopes. This made it much easier to reason about the lifetime of objects and write types and programs that can enlist the help of the compiler to ensure that resources are cleaned up, even in the presence of complicated control flow - like exceptions.

The combination of structured lifetime and structured control-flow and tying them both to the same program scopes makes it much easier to reason about the code.

Structured concurrency revolves around the idea that (potentially) concurrent execution can be treated as a resource with a lifetime, much like how objects can have lifetimes.

A potentially concurrent operation can be actively using resources it is given a reference to, and so the basic principle is that we need to make sure that the end of the lifetime of the concurrent operation “happens before” the destruction of any resources it might be accessing.

The way do this is by ensuring that all potentially concurrent operations that might be accessing a given resource are “joined” before that resource is destroyed (i.e. when it goes out of scope).

If we fail to join a potentially concurrent operation before resources it's using goes out of scope then the program has undefined behaviour. Further, if we create a detached operation (work that can *never* be joined) then this makes it very difficult, and in some

cases impossible, to know when it will be safe to destroy resources passed by-reference to that operation.

Fire and forget interfaces, such as the proposed `std::execution::execute()` from P0443R14, often require the programmer to manually ensure that they do some extra work at the end of the operation to signal completion so that the work can be joined.

This tends to lead to unstructured, ad-hoc mechanisms for signalling completion and joining concurrent work. Further, this logic is often mixed in with business logic for the operation and tends to be repeated each time a given concurrency pattern is used, leading to code that is more error-prone and hard to maintain.

§ 2.2.1 Garbage collection

One traditional approach to ensuring the correct order of destruction of resources used by concurrent programs is to use some form of garbage collection.

In garbage-collected languages, such as C#, the runtime ensures that the lifetimes of objects are maintained as long as they are reachable by some thread.

In C++ programs it is not uncommon for asynchronous code to make heavy use of `std::shared_ptr` to ensure that resources are kept alive until an async operation completes.

Much of the the asynchronous code at Facebook written using `folly::Future`-based APIs makes use of `std::shared_ptr`, usually captured within a lambda callback, to ensure that objects are kept alive until an asynchronous operation completes.

There are several downsides to using `std::shared_ptr` for this:

- Garbage collection approaches like `std::shared_ptr` tend to be viral, often requiring classes to inherit from `std::shared_from_this` or to hold child objects by `std::shared_ptr`.
- The context in which the destruction occurs becomes non-deterministic. In cases where there might be a race between multiple threads releasing the last reference then the thread on which the object is destroyed can change from run-to-run, making it difficult to reason about the code. In some cases, this non-determinism can be a correctness issue, e.g. for objects that must be destroyed from a certain thread.
- It forces heap-allocation and synchronisation (for the atomic refcount) which can impact the performance of an application.

§ 2.2.2 Coroutines

Coroutines provide a language representation that allows developers to write asynchronous code that looks like normal code in terms of control-flow and lifetime. You can declare local variables, just like with normal functions, and the compiler makes sure that these variables are destroyed when control-flow exits the scope of that object.

However, this automatic destruction of local variables on scope-exit means that we need to ensure that we join any concurrent operations that may be holding a reference to those local variables before they go out of scope.

For example:

```
task<void> do_something(const resource& r);

task<void> example() {
    ...
    {
        resource r;
        co_await do_something(r);
    }
    ...
}
```

In this instance, we are calling an asynchronous child operation, `do_something()` and passing it a reference to a resource object that is a local variable in the calling coroutine.

Passing parameters by reference in this fashion is a natural way to write code as it mirrors very closely the way in which we write synchronous code.

When the `co_await` expression completes and the awaiting coroutine is resumed, execution will then exit the scope of the object, `r`, and it will be destroyed.

For this code to avoid running into undefined behaviour, it needs to ensure that any concurrent operations created within the `do_something()` operation that might be accessing `r` have all completed before it resumes the `example()` coroutine and `r` is destroyed.

To support this, we need `do_something()` to ensure that it provides the structured concurrency guarantee. i.e. that it does not leak any detached work that might still be accessing `r` after `do_something()` completes.

To expand on this further, let's look at a possible implementation of `do_something()`.

```
task<A> do_part1(const resource& r);
task<B> do_part2(const resource& r);
void do_part3(const A& a, const B& b)

task<void> do_something(const resource& r) {
    auto [a, b] = co_await when_all(
        do_part1(r),
        do_part2(r));

    do_part3(a, b);
}
```

In this case, the `do_something()` operation is launching two potentially concurrent child operations, `do_part1()` and `do_part2()`, and then processing the results.

Both of these operations must complete successfully to be able to combine their results. However, if one of them fails with an error then we no longer need the results of the other operation since the operation as a whole will complete with an error.

In this case, we could theoretically imagine that the `when_all()` operation might be able to complete early with an error if the first operation fails, since at that point we know what its result will be.

However, if we don't also wait for the other operation to complete then the `do_something()` operation may run to completion and then resume the calling `example()` coroutine which will then destroy the resource object, `r`, before the other child task completes.

Thus to avoid a dangling reference, the `when_all()` implementation still needs to ensure that all child-operations have run to completion, even though it has already computed its result and the results of the outstanding operations will just be discarded.

In general, the ability to pass parameters by reference to coroutines safely *requires* that we ensure concurrent child operations are all joined before returning..

If we implement algorithms with this property then when we compose them, those higher level algorithms can also have this property. If any algorithms fail to preserve this “structured concurrency” property then any consumers of that algorithm will generally also fail to preserve that property.

Returning to the `when_all()` use-case above, it still has to wait for the other operation to finish if the first one fails early with an error. If it were to just wait for the operation to complete naturally then the operation as a whole will take longer to complete than necessary.

Ideally we'd have some way to ask that remaining child operation to complete as soon as it can because we are just going to discard its result. i.e. we need a way to request cancellation of that other operation.

§ 3 Prior-art in cancellation patterns

This section looks at some existing cancellation patterns used in C++ and in other languages to understand some of the limitations.

§ 3.1 Networking TS / `boost::asio` cancellation model

Many of the asynchronous facilities within `boost::asio` have support for cancellation.

The ability to request cancellation of operations is generally exposed to the user as a `.cancel()` method on the I/O object that was used to launch the asynchronous operation.

For example, the `basic_waitable_timer` type described in [N4734](#) (C++ Extensions for Networking Working Draft) provides a `.cancel()` method that will cancel all outstanding `async_wait()` operations launched on the timer object.

For example:

```
using namespace std::experimental::net;

io_context& ioCtx = ...;

system_timer timer{ioCtx};
timer.expires_after(500ms);
timer.async_wait([](std::error_code ec) {
    if (!ec) {
        std::cout << "500ms has elapsed!";
    } else if (ec == std::errc::operation_canceled) {
        std::cout << "timer was cancelled\n";
    } else {
        std::cout << "error: " << ec.message() << "\n";
    }
});

// ... later

// Request cancellation of async_wait() operations.
// Causes async_wait() to complete with std::errc::operation_canceled
size_t numberCancelled = timer.cancel();
```

The `basic_waitable_timer` class-template also supports a `.cancel_one()` method that cancels at most one single subscribed `async_wait()` operation rather than cancelling all subscriptions.

Other I/O objects in the Networking TS also provide a similar cancellation interface. For example, the `basic_socket` class-template provides both a `void cancel()` method, as well as a `void cancel(error_code& ec)` method - the latter providing the ability to report an error on a failure to request cancellation.

Both of these `.cancel()` methods cancel all outstanding asynchronous operations associated with the socket object and cause their completion-handlers to be passed an error code that matches `std::errc::operation_canceled`.

Similarly, the `basic_socket_acceptor` and `basic_socket_resolver` class-templates also provide `void cancel()` and `void cancel(error_code& ec)` methods that cancel all outstanding asynchronous operations.

It is also worth noting that the I/O objects do not support concurrent access and so it is the caller's responsibility to ensure that any calls to `.cancel()` are serialised with respect to calls to other operations on the I/O object.

The typical approach to ensuring this requirement is satisfied when the source of a cancellation request occurs on another thread/executor is to schedule the call to `.cancel()`

onto the I/O object's associated strand executor.

One needs to be careful, then, to ensure that the I/O object is not destroyed in the meantime while the call to `.cancel()` is in the executor's queue waiting to be processed.

§ 3.1.1 Some Limitations

While this approach to cancellation has worked successfully for users of `boost::asio` and the Networking TS in the domain of networking, there are a few limitations to this approach that make it difficult to generalise to all async operations.

It is also interesting to consider why things are designed this way.

Inability to cancel an individual operation

The semantics of the `.cancel()` methods is to cancel all pending asynchronous operations on the associated I/O object.

This makes it impossible to cancel just one specific asynchronous operation.
e.g. cancelling the `async_read_some()` on a socket when there is also an outstanding `async_write_some()` on the same socket.

The ability to cancel individual operations may become more important if/when async facilities are extended to async file I/O.

For example, consider a database application that may have issued many concurrent read operations on a file, with each read associated with a different query. If some queries are cancelled then ideally only the read operations associated with those queries would be cancelled - not all read operations on the file.

A cancellation design that requires the program to cancel all outstanding operations associated with a given object would not work well for many use-cases.

Requires an I/O object

The ability to cancel async operations requires direct access to the I/O object that the operations are associated with.

While this generally maps closely to the requirements of the underlying OS calls for async I/O, this pattern of cancellation does not generalise well to asynchronous operations that are not methods of objects.

For example, say we want to write a high-level asynchronous operation that downloads a file over the network.

e.g.

```
template<typename CompletionToken>
auto download_file(io_context& ctx,
                  std::string url,
```



```
std::string path,  
CompletionToken ct);
```

This high-level algorithm might completely encapsulate creating and managing the lifetime of any sockets needed to communicate over the network internally, and never provide access to these to the caller.

The lack of any I/O object in this style of interface means that we have nothing that we can call `.cancel()` on to cancel all associated operations.

While the API could instead be exposed as a method on a hypothetical `file_downloader` class that acted as an I/O object and that kept track of outstanding async-operations and provided a `.cancel()` method on that object to cancel the requests, this approach makes building and using algorithms that support cancellation more cumbersome.

The granularity of cancellation being on the I/O object level also makes it difficult for generic concurrency algorithms that inject cancellation to limit their effect to only those operations that they are composing.

Requires that every I/O operation supports cancellation

The design of types such as `basic_waitable_timer` and `basic_socket` is such that the `.cancel()` method needs to be able to cancel all outstanding I/O operations associated with those I/O objects.

This means that, logically, the implementation needs to keep track of all outstanding, associated async operations so that they can be cancelled when the `.cancel()` method is called.

In general, this would tend to imply an increased amount of complexity in the implementation of these classes, as they need extra data-members to be able to store a list of the operations.

Whether this adds overhead in practice depends on the underlying platform APIs on which these types are mapping to.

For Windows IOCP-based sockets, the OS keeps track of the set of outstanding operations for each file-handle and calling `CancelIoEx(fileHandle, NULL)` will cancel all of them. So there is no overhead in the implementation to support this on Windows.

For a Linux `epoll`-based socket implementation, the list of outstanding operations needs to be maintained anyway as the event notification is per-file-handle / operation-type combination rather than per-async operation. Upon receipt of a notification from the OS that the socket is now readable, the implementation needs to be able to iterate through the list of waiting `async_read_some()` operations to retry them.

The same lists of operations can be traversed by `.cancel()` when cancelling all outstanding I/O operations. Also, cancelling all of the operations at once can have a slight

performance benefit compared to cancelling each operation individually as synchronisation is required to lock the internal data-structures in some cases and cancelling all operations can do this synchronisation once instead of once per operation.

A Linux `io_uring`-based socket implementation would be similar to the Windows IOCP in that `io_uring` provides notification on completion of individual async I/O operations and so does not intrinsically require the I/O object to maintain a list of outstanding async operations itself in the same way an `epoll` implementation does. However, it would be required to maintain this list in order to support a `.cancel()` method that cancels all outstanding operations as each individual `io_uring` operation must be cancelled separately. There would therefore be some additional complexity/overhead to support this.

The restriction that the I/O object must not be accessed concurrently from multiple threads and that calls to `.cancel()` must be sequenced with respect to other calls that start async operations avoids any synchronisation overhead that would be required if cancellation were able to be requested concurrently from other threads. This synchronisation burden is typically placed instead on the strand executor used to serialise access to the object.

In general, however, the pattern of supporting cancellation of all async operations associated with a given I/O object can result in additional bookkeeping needed to maintain the list of operations so that they can be cancelled.

Also, as a given async-operation does not necessarily know at the time it is launched whether or not it will be cancelled by a subsequent call to `.cancel()`, the implementation must assume it might and so perform the necessary bookkeeping to allow this, even if the operation will never be cancelled. There is no way for a caller to specify that this particular async-operation will never be cancelled.

Lack of generic composability

The Networking TS design does not require every async operation or I/O object to implement the same interface for cancellation - some have a `.cancel()` and a `.cancel_one()` which return a count, while others have a `.cancel()` and `.cancel(error_code&)`.

The lack of a uniform interface to I/O objects and cancellation in general makes it impossible to compose arbitrary async operations generically with general-purpose algorithms that need to be able to request cancellation.

For example, a generic `when_all()` or `timeout()` algorithm written to work with any child async operations cannot cancel those child operations unless there is a uniform/generic interface that it can be implemented in terms of.

This often leaves applications that compose Networking TS-based async operations and that want to support cancellation having to do so in an ad-hoc manner.

§ 3.2 .NET Framework cancellation model

The .NET Framework has had support for cancellation of async operations since the async model was greatly expanded in .NET v4.0.

Cancellation in the .NET Framework is generally built on top of the `CancellationToken`, `CancellationTokenSource` and `CancellationTokenRegistration` family of classes. These types closely correspond to the `std::stop_token` family of classes added to the C++ standard library in C++20.

Cancellable operations in .NET generally accept an additional parameter of type `CancellationToken`.

Many APIs also provide overloads that do not take a `CancellationToken` to make it easier to call in cases where the caller does not require cancellation. These APIs often just forward to the `CancellationToken`-taking version, passing `CancellationToken.None` - which is a token for which cancellation will never be requested.

For example:

```
namespace System.Net.Http
{
    class HttpClient
    {
        ...

        // Non-cancellable version
        public Task<HttpResponseMessage> GetAsync(Uri requestUri) {
            return this.GetAsync(requestUri, CancellationToken.None);
        }

        // Cancellable version
        public Task<HttpResponseMessage> GetAsync(Uri requestUri,
                                                    CancellationToken
                                                    cancellationToken);

        ...
    }
}
```

The implementation of cancellable async operations can either poll the token to see if cancellation has been requested by querying the `IsCancellationRequested` property, or can subscribe a callback to receive notification of a cancellation request using the `CancellationTokenRegistration` type.

The implementation of a cancellable async operation can also query whether or not cancellation will ever be requested by querying the `.CanBeCancelled` property. Some async operations can use a more efficient strategy if they don't have to worry about supporting cancellation.

This is something that the Networking TS cancellation model above is not able to do.

The caller can request cancellation by creating a new `CancellationTokenSource` object and passing the `CancellationToken` obtained from its `.Token` property, then later calling `.Cancel()` method on the `CancellationTokenSource`.

Task eagerness impacts cancellation design

Note that the way Task-returning async methods work in the .NET Framework is that they start executing immediately when they are called and only return the `Task` object to the caller when they suspend, i.e. at an `await` expression, or when the method completes, either by returning a value or exiting via an exception.

For example:

```
Task Example() {
    Part1();
    await Part2();
    Part3();
}

Task Caller() {
    Task t = Example();
    // By here 'Example()' has already called Part1() and Part2()
    // and is now suspended in 'await' expression.
}
```

One of the main reasons that .NET chose this eager execution model is to avoid heap-allocating storage for the local-variables in cases where the operation completes synchronously.

Instead of immediately heap-allocating storage for the async state, the implementation of an `async`-qualified method initially allocates the async state on the stack and then lazily moves that state to a heap-allocation only if the method suspends at an `await` expression.

This reduces the number of short-lived heap-allocations and thus reduces the frequency with which the garbage collector needs to run - at least for `async` function invocations that complete synchronously.

This is something that is possible because of the garbage collected nature of .NET that allows relocating objects in memory.

The same strategy does not work for coroutines in C++ because the state can have internal pointers which cannot in general be automatically updated if the state is relocated.

The fact that `async` functions execute eagerly then requires that the `CancellationToken` is available at the time the function is invoked so that it can be referenced during that initial sequential part of its execution, but also so that it can be passed down to other child tasks.

Thus, as the `CancellationToken` needs to be available at the time the operation starts executing, it *must* be provided as an argument to the function.

Inability to inject cancellation

One implication of this requirement to pass the `CancellationToken` as an argument when a Task-returning method is called, however, is that it means that you cannot use higher-order functions to compose Task objects and allow those higher-order functions to inject cancellation into those tasks.

For example, the `Task.WhenAll(params Task[] tasks)` algorithm is passed an array of Task objects and returns a Task that completes when all of the input tasks have completed. However, this algorithm is not able to request cancellation of the other Task objects if one of the tasks fails with an exception as it has no way to inject a `CancellationToken` into those async operations that it can use to communicate the request.

To be able to compose cancellation patterns generically, you would instead need to pass a lambda that takes a `CancellationToken` and returns a Task into the higher-order function so that the algorithm could inject its own `CancellationToken`.

The inability to compose cancellation patterns generically using higher-order algorithms in terms of Task will tend to mean that common concurrency/cancellation patterns will need to be manually re-implemented for each composition of child operations.

Taking a CancellationToken argument advertises cancellability

One of the interesting outcomes of the design choice for cancellation in .NET is that by requiring the caller to pass a `CancellationToken` as a parameter to cancellable async functions is that it advertises cancellability of that operation - it's right there in the signature!

It also forces the implementation to think about cancellation. This can be a good thing as it means cancellability is more likely to be implemented. But it can also be a bad thing, as it forces the author of the code to do the busy-work of ensuring the `CancellationToken` is plumbed through to any child operations.

Passing a CancellationToken tends to work well for await-style asynchrony

With traditional callback-style async programming we often make a call to launch an operation and then execution returns to that caller which can then do something else, such as storing a handle to the operation somewhere that can be later used to request cancellation of that operation.

For example, code using `folly::Future`-based APIs might do the following:

```
folly::Future<int> an_operation();

void some_class::start_the_operation() {
    // Launch an async operation.
```

```

// Store the handle somewhere we can access it later.
this->future = on_operation();

// Attach a callback to be run when the operation completes.
this->future.addCallback_([&](folly::Try<int>&& result) {
    if (result.hasValue()) {
        // Succeeded
    } else if (result.hasException<folly::OperationCancelled>()) {
        // Cancelled
    } else {
        // Other error
    }
});

// do something else that might trigger a cancellation...
}

void some_class::on_some_event() {
    // Request cancellation of the operation by calling .cancel()
    // on the folly::future. The callback attached via setCallback_()
    // will still be run when the operation completes.
    this->future.cancel();
}

```

However, when writing coroutine-based async code we often immediately await the handle returned by invoking another coroutine.

For example:

```

Task<int> AnotherOperation();

Task<void> Consumer() {
    // Immediately awaiting the returned task has two implications:
    // - it consumes the task, meaning we cannot subsequently use it to
    //   request cancellation.
    // - it suspends execution of the calling coroutine, meaning it is
    //   unable to execute any other code that might cancel the operation.
    int x = await AnotherOperation();

    // Operation already completed here. No opportunity to cancel it.
}

```

To allow the caller to communicate a cancellation request it can pass a `CancellationToken` (or `std::stop_token` in C++) into the child operation to allow the caller to communicate a request to cancel and for the child operation to receive that request.

In the `cppcoro` cancellation model and the .NET Framework cancellation model, the cancellation-token is usually passed as a parameter. This works well and is generally composable when making immediately invoked/awaited coroutine calls.

e.g.

```
using namespace cppcoro;

task<int> another_operation(cancellation_token ct);

task<void> consumer(cancellation_token ct) {
    // Explicitly passing the cancellation-token to child operations allows
    // us to communicate cancellation requests of the parent operation
    // through
    // to child operations.
    int x = co_await another_operation(ct);

    // use x
}
```

§ 3.3 cppcoro::task cancellation

The `cppcoro` library takes a similar approach to cancellation that the .NET Framework takes.

Async operations that are cancellable can optionally take an extra `cancellation_token` parameter which the caller can use to later communicate a request to cancel the operation.

This approach suffers from the same composability limitations discussed in the section above on the .NET Framework cancellation model.

A key limitation here is that we cannot compose operations represented by `cppcoro::task` objects using generic concurrency algorithms and have those algorithms manage cancellation of the child operations.

The workaround has been to have these generic algorithms invoked with task factories that allow the algorithm to inject a `cancellation_token` that it controls.

For example, a generic `timeout()` algorithm:

```
using namespace cppcoro;

template<
    typename AwaitableFactory,
    typename Scheduler,
    typename Duration,
    typename Result = await_result_t<invoke_result_t<AwaitableFactory&,
        cancellation_token>>>
task<Result> timeout(AwaitableFactory f,
                    Scheduler s,
                    Duration d,
                    cancellation_token ct = {}) {
    cancellation_source src;
    cancellation_registration cancelParent{std::move(ct),
                                          [&] { src.request_cancellation();
                                          }
    };
    auto [result, _] = co_await when_all(
        [&]() -> task<Result> {
```

```

    auto cancelOnExit = on_scope_exit([&] { src.request_cancellation();
    });
    co_return co_await f(src.token());
}(),
[&]() -> task<void> {
    auto cancelOnExit = on_scope_exit([&] { src.request_cancellation();
    });
    try {
        co_await s.schedule_after(d, src.token());
    } catch (const operation_cancelled&) {}
}());
co_return std::move(result);
}

```

The usage of such an algorithm is cumbersome as you end up having to wrap every call to an async operation that you want to compose with concurrency algorithms in a lambda that lets you inject the appropriate `cancellation_token`.

```

using namespace cppcoro;

task<response> query(request req, cancellation_token ct = {});
static_thread_pool threadPool;

task<void> example_usage(cancellation_token ct = {}) {
    request req;
    req.set_param(123);
    //...

    // Simple call without timeout() is relatively straigh-forward.
    response result1 = co_await query(req, ct);

    // Wrapping with a generic timeout() algorithm is painful.
    response result2 = co_await timeout([&](cancellation_token ct) {
        return query(std::move(req), std::move(ct));
    }, threadPool.get_scheduler(), 500ms, ct);
}

```

This pattern generally scales poorly to large applications if you want to make use of generic concurrency algorithms that deal with cancellation.

§ 3.4 folly::coro cancellation model

The `folly::coro` library, which is the coroutine abstraction layer used within much of Facebook C++ code, takes a different approach to cancellation that greatly reduces the boiler-plate needed for handling cancellation.

Like `cppcoro` and the .NET Framework, `folly::coro` cancellation is build on a `CancellationToken`-based abstraction. The `folly::coro::Task` coroutine type is different, however, in that it is by-default transparent to cancellation.

Each `folly::coro::Task` has an implicitly associated `CancellationToken` that is automatically injected into any child operation that is `co_awaited` by that coroutine.

For example: Simple usage implicitly passes CancellationToken to child tasks.

```
folly::coro::Task<Response> query(Request req);

folly::coro::Task<void> example_usage() {
    Request req;
    req.setParam(123);
    // ...

    // Simple call is even simpler.
    Response result1 = co_await query(req);

    // Wrapping call with a generic timeout() algorithm is much simpler.
    // This will cancel query() when either example_usage() is cancelled or
    // when the timeout expires.
    Response result2 = co_await folly::coro::timeout(query(req), 500ms);
}
```

Having the CancellationToken implicitly passed down to child operations removes much of the burden of manually plumbing cancellation parameters, greatly simplifying writing cancellation-correct code with coroutines.

A coroutine can obtain the current CancellationToken as follows:

```
folly::coro::Task<void> current_token_example() {
    const folly::CancellationToken& ct =
        co_await folly::coro::co_current_cancellation_token;

    // later ...

    // Poll for cancellation
    if (ct.isCancellationRequested()) {
        // Exit early.
        co_yield folly::coro::co_error(folly::OperationCancelled{});
    }

    // Or attach a CancellationCallback
    {
        auto handle = startSomeOperation();

        folly::CancellationCallback cb{ct, [&]() noexcept {
            cancelOperation(handle);
        }};

        // Wait until cancellation is requested.
        co_await waitForOperation(handle);
    }
}
```

You can manually inject your own CancellationToken to allow you to request cancellation of a child operation by manually calling the `co_withCancellation()` function. This overrides the implicit CancellationToken from the parent coroutine.

```

folly::coro::Task<void> do_something();

folly::coro::Task<void> manual_override_example() {
    folly::CancellationSource cancelSrc;

    // Manually hook up the parent coroutine's CancellationToken to
    // forward through to 'cancelSrc'.
    folly::CancellationCallback cancelWhenParentCancelled{
        co_await folly::coro::co_current_cancellation_token,
        [&]() noexcept { cancelSrc.requestCancellation(); }
    };

    // Inject a different CancellationToken in to the child
    co_await folly::coro::co_withCancellation(cancelSrc.getToken(),
        do_something());
}

```

The implementation of `folly::coro::Task` works as follows:

- The Task's promise object holds the coroutine's associated `CancellationToken` as a data-member.
- The Tasks' coroutine type uses the `await_transform()` mechanism to automatically apply the `co_withCancellation()` customisation-point to the argument of every `co_await` expression in the coroutine-body - this is invoked with the current `CancellationToken` and the awaitable.
- Awaitable types can customise `co_withCancellation()` to handle injecting the `CancellationToken` and thus opt-in to support for cancellability.
- The Task type itself customises `co_withCancellation()` to inject the parent coroutine's `CancellationToken` into the child coroutine's promise.

A simplified sketch of how the implementation fits together:

```

template<typename T>
class TaskPromise {

    // Every co_await expression injects the CancellationToken by applying
    // co_withCancellation() to the operand to the co_await operator,
    // passing the parent coroutine's current Cance
    template<typename T>
    decltype(auto) await_transform(T&& value) {
        return folly::coro::co_withCancellation(cancelToken_, static_cast<T&&>
            (value));
    }

private:
    folly::CancellationToken cancelToken_;
    bool hasCancelToken_ = false;
    ...
};

template <typename T>
class Task {
public:

```

```

using promise_type = TaskPromise<T>;
...

private:
    friend Task co_withCancellation(const CancellationToken& cancelToken,
                                   Task&& task) noexcept {
        auto& promise = task.coro_.promise();

        // Don't override previous CancellationToken if set.
        if (!promise.hasCancelToken_) {
            task.coro_.promise().cancelToken_ = cancelToken;
            hasCancelToken_ = true;
        }
        return std::move(task);
    }

    std::coroutine_handle<promise_type> coro_;
};

```

This mechanism for automatically passing the `CancellationToken` to child operations relies on the fact that `Task`-returning coroutines are lazily started only when the returned task is awaited. This allows us to safely inject the `CancellationToken` into the child coroutine as part of the `co_await` expression, just before the child coroutine is launched. It also allows passing these tasks to higher-order concurrency algorithms that can then create their own cancellation scopes and inject their own `CancellationToken` that they can use to request cancellation.

This would not be possible if the task was started eagerly when the coroutine function was first invoked.

Benefits

The ability to have cancellation-tokens implicitly passed through by the coroutine mechanics greatly simplifies a lot of application code that generally only needs to be transparent to cancellation.

Cancellation is generally either requested by high-level handlers (e.g. RPC request framework request might request cancellation if the connection is dropped) or by general-purpose concurrency algorithms that introduce new cancellation-scopes (e.g. a `timeout()` algorithm). A handful of leaf operations can then be built that respond to cancellation (e.g. RPC requests, timers, etc.)

This allows centralising the handling of cancellation to a relatively small fraction of the code-base with the bulk of the application code supporting cancellation without needing to write any additional code to do so.

Limitations

With this approach, where every `Task` always has a (possibly null) `CancellationToken`, we cannot statically determine whether cancellation will be requested and thus cannot

statically eliminate all overhead related to cancellation support. However, we can still determine at runtime whether or not cancellation might be requested by calling `.canBeCancelled()` on the `CancellationToken`.

One example of the runtime overhead this adds is that we need an extra pointer of storage for every coroutine frame to store the `CancellationToken` as it is passed down.

The general-purpose nature of `CancellationToken` requires allocating some shared-storage on the heap with the lifetime managed through use of atomic reference-counting. This allows it to be used safely in a wide variety of scenarios.

However, for many of the coroutine scenarios we have a structured concurrency model where the `CancellationToken` passed to child coroutines is never used after those child coroutines complete. Also, there are cases where a coroutine creates a new `CancellationSource` as a local variable and never moves or copies it.

A more efficient implementation that takes advantage of this more restrictive, structured use of cancellation-tokens could potentially avoid the allocation and reference counting by using a different cancellation-token type that allocates the shared-state inline in the cancellation-source object and has cancellation-tokens simply hold a non-reference-counted pointer to this shared-state.

Finally, the mechanism used to propagate the cancellation context from the parent coroutine to child coroutines is currently hard-coded. It calls the cancellation-specific CPO `co_withCancellation()` and uses hard-coded `CancellationToken` type. However, there may be other kinds of context that an application wants to propagate automatically to child coroutines using similar mechanics to the cancellation-token propagation. It's possible that this facility can be generalised to support passing other kinds of context through implicitly. e.g. an allocator, executor, logging context.

§ 3.5 `std::stop_token`

C++20 added three new types to the standard library: `std::stop_token`, `std::stop_source` and `std::stop_callback<CB>`.

These types were added to C++20 to support cancellation as part of the `std::jthread` abstraction and have some integration with the `std::condition_variable_any` wait-functions.

Example: A simple usage of `jthread/stop_token/condition_variable_any`

```
#include <mutex>
#include <thread>
#include <stop_token>
#include <condition_variable>

int main() {
    std::jthread worker{[](std::stop_token st) {
        std::condition_variable_any cv;
```

```

std::mutex m;
std::unique_lock lk{m};
for (int i = 0; !st.stop_requested() ; ++i) {
    std::cout << "tick " << i << std::endl;

    // An interruptible sleep_for()
    cv.wait_for(lk, st, 100ms, std::false_type{});
}
};

// Do something on the main() thread
std::this_thread::sleep_for(1s);

// When 'worker' goes out of scope its destructor will call
// worker.request_stop() before joining the thread.
// This will communicate the request to stop via the std::stop_token
// passed to the thread's entry-point function and this will interrupt
// the cv.wait_for() call and cause the thread to exit promptly.
}

```

However, the `stop_token` abstraction is more general than `std::jthread` and can also be used for cancellation of asynchronous code in much the same way that `CancellationToken` is used in `folly::coro` and in the .NET Framework.

A general purpose cancellation mechanism for sender/receiver would ideally integrate with the `std::stop_token` abstraction. Although it is worth noting that the design of `std::stop_token`, like `folly::CancellationToken`, requires a heap-allocation and reference-counting, which is not strictly necessary for structured concurrency use-cases.

§ 4 Proposed Changes

This section will describe the facilities being proposed by this paper. The following section, ‘Design Discussion’ will discuss some of the design considerations relating to this proposal.

§ 4.1 Add `stoppable_token` and related concepts

Add the following concept definition to the `<concepts>` header:

```

namespace std
{
    template<template<typename> class>
    struct __check_type_alias_exists; // exposition-only

    template<typename T>
    concept stoppable_token =
        copy_constructible<T> &&
        move_constructible<T> &&
        is_nothrow_copy_constructible_v<T> &&
        is_nothrow_move_constructible_v<T> &&
        equality_comparable<T> &&

```

```

requires (const T& token) {
    { token.stop_requested() } noexcept -> boolean-testable;
    { token.stop_possible() } noexcept -> boolean-testable;
    typename __check_type_alias_exists<T::template callback_type>;
};

template<typename T, typename CB, typename Initializer = CB>
concept stoppable_token_for =
    stoppable_token<T> &&
    invocable<CB> &&
    requires {
        typename T::template callback_type<CB>;
    } &&
    constructible_from<CB, Initializer> &&
    constructible_from<typename T::template callback_type<CB>, T,
        Initializer> &&
    constructible_from<typename T::template callback_type<CB>, T&,
        Initializer> &&
    constructible_from<typename T::template callback_type<CB>, const T,
        Initializer> &&
    constructible_from<typename T::template callback_type<CB>, const T&,
        Initializer>;

template<typename T>
concept unstoppable_token =
    stoppable_token<T> &&
    requires {
        { T::stop_possible() } -> boolean-testable;
    } &&
    (!T::stop_possible());
}

```

The `stoppable_token` concept checks for the basic interface of a “stop token” which is copyable and allows polling to see if stop has been requested and also whether a stop request is possible.

It also provides an associated nested template-type-alias, `T::callback_type<CB>`, that identifies the stop-callback type to use to register a callback to be executed if a stop-request is ever made on a `stoppable_token` of type, `T`.

The `stoppable_token` concept has a number of semantic requirements on types:

- All copies of a `stoppable_token` reference the same logical shared stop state and shall report values consistent with each other.
- Given a token, `t`, if `t.stop_possible()` evaluates to `false` then for any token, `u`, that references the same logical shared stop state, `u.stop_possible()` shall also subsequently evaluate to `false` and `u.stop_requested()` shall also subsequently evaluate to `false`.
- Given a token, `t`, if `t.stop_requested()` evaluates to `true` then for any token, `u`, that references the same logical shared stop state, `u.stop_requested()` shall also subsequently evaluate to `true` and `u.stop_possible()` shall also subsequently evaluate to `true`.

- Given a token, `t`, of type `T`, a callback-type, `CB`, and a callback-initializer argument, `init`, of type `Initializer` then constructing an instance, `cb`, of type `T::callback_type<CB>`, passing `t` as the first argument and `init` as the second argument to the constructor, shall, if `t.stop_possible()` is true, construct an instance, `callback`, of type `CB`, direct-initialized with `init`, and register `callback` with `t`'s shared stop state such that `callback` will be invoked with an empty argument list if a stop request is made on the shared stop state.
 - If `t.stop_requested()` is true at the time `callback` is registered then `callback` *may* be invoked immediately inline inside the call to `cb`'s constructor.
 - If `callback` is invoked then for any token, `u`, that references the same shared stop state as `t`, an evaluation of `u.stop_requested()` will be true if the beginning of the invocation of `callback` strongly-happens-before the evaluation of `u.stop_requested()`.
 - If `t.stop_possible()` evaluates to false then the construction of `cb` is not required to construct and initialize `callback`.
- Construction of a `T::callback_type<CB>` instance shall only throw exceptions thrown by the initialization of the `CB` instance from the value of type `Initializer`.
- Destruction of the `T::callback_type<CB>` object, `cb`, deregisters `callback` from the shared stop state such that `callback` will not be invoked after the destructor returns.
 - If `callback` is currently being invoked on another thread then the destructor of `cb` will block until the invocation of `callback` returns such that the return from the invocation of `callback` strongly-happens-before the destruction of `callback`.
 - Destruction of a callback `cb` shall not block on the completion of the invocation of some other callback registered with the same shared stop state.

§ 4.2 Tweaks to `std::stop_token`

Modify `std::stop_token` to add the nested `callback_type` template type-alias.

```
namespace std
{
    class stop_token {
    public:
        template<typename T>
        using callback_type = stop_callback<T>;

        // ... remainder of stop_token definition as before
    };
}
```

This type alias is required for the `std::stop_token` type to satisfy the `stoppable_token` concept.

§ 4.3 Add `std::never_stop_token`

The `never_stop_token` type implements the `unstoppable_token` concept. i.e. `stop_possible()` and `stop_requested()` are static constexpr member functions that always return false.

```
// <stop_token> header

namespace std
{
    class never_stop_token {
        // exposition only
        class callback {
        public:
            template<typename C>
            explicit callback_type(never_stop_token, C&&) noexcept {}
        };
        public:
            template<invocable CB>
            using callback_type = callback;

            static constexpr bool stop_requested() const noexcept { return false;
            }
            static constexpr bool stop_possible() const noexcept { return false; }
        };
    }
```

This can be returned from `get_stop_token()` customisation-point to indicate statically that you will never submit a stop-request to the operation - this is the default behaviour of the `get_stop_token()` customisation-point.

Child operations that attempt to use this type as a stop-token to detect and respond to cancellation requests will generally optimise out those code-paths and avoid using any storage for the stop-callback.

Note that `unstoppable_token<never_stop_token>` will evaluate to true. This trait should be used to detect never-cancellable use-cases instead of testing for `same_as<never_stop_token>` to allow other libraries to also define stop-token types that may be unstoppable.

§ 4.4 Add `std::in_place_stop_token`, `std::in_place_stop_source`, `std::in_place_stop_callback<CB>`

The `in_place_stop_token` type implements the `stoppable_token` concept, similarly to `stop_token`, but places more restrictions on its usage and in doing so permits a more efficient implementation that does not require heap allocations or reference-counting to manage the lifetime of the shared stop-state.

Instead, the shared stop-state is stored inline inside the `in_place_stop_source` object and its lifetime is tied to the lifetime of that object. The `in_place_stop_token` objects can then just hold a raw pointer to the shared stop-state and copying the stop-token objects no

longer requires atomic ref-counting. However, this then means that applications must ensure all usage of `in_place_stop_token` and `in_place_stop_callback` objects occurs prior to the invocation of the destructor of the associated `in_place_stop_source`.

These restrictions match the typical use-cases for sender-based structured concurrency algorithms that need to introduce new cancellation scopes. And by carefully placing semantic constraints on usages of the `get_stop_token()` customisation-point (described below) we can enforce that all usages of stop-tokens used for cancellation in sender-based operations are constrained to work within the limitations of the `in_place_stop_token` type.

This proposal adds the following to the `<stop_token>` header:

```
namespace std
{
    class in_place_stop_token;
    template<typename CB>
    class in_place_stop_callback;

    class in_place_stop_source
    {
    public:
        in_place_stop_source() noexcept;
        ~in_place_stop_source();

        // Not copyable/movable
        in_place_stop_source(const in_place_stop_source&) = delete;
        in_place_stop_source(in_place_stop_source&&) = delete;
        in_place_stop_source& operator=(const in_place_stop_source&) = delete;
        in_place_stop_source& operator=(in_place_stop_source&&) = delete;

        bool request_stop() noexcept;

        [[nodiscard]] bool stop_requested() const noexcept {
            return stop_requested_flag.load(std::memory_order_acquire); //
                exposition-only
        }

        [[nodiscard]] in_place_stop_token get_token() const noexcept;

    private:
        atomic<bool> stop_requested_flag{false}; // exposition-only
    };

    class in_place_stop_token
    {
    public:
        template<typename CB>
        using callback_type = in_place_stop_callback<CB>;

        in_place_stop_token() noexcept
        : src(nullptr) // exposition-only
        {}
    };
}
```

```

in_place_stop_token(const in_place_stop_token& other) noexcept
: src(other.src) // exposition-only
{}

[[nodiscard]] bool stop_possible() const noexcept {
    return src != nullptr; // exposition-only
}

[[nodiscard]] bool stop_requested() const noexcept {
    return src != nullptr && src->stop_requested(); // exposition-only
}

friend [[nodiscard]] bool operator==(in_place_stop_token a,
                                     in_place_stop_token b) noexcept {
    return a.src == b.src; // exposition-only
}

void swap(in_place_stop_token& other) noexcept {
    std::swap(src, other.src); // exposition-only
}

friend void swap(in_place_stop_token& a, in_place_stop_token& b)
    noexcept {
    a.swap(b);
}

private:
    in_place_stop_source* src; // exposition-only
};

template<typename CB>
class in_place_stop_callback {
public:
    template<typename Initializer>
        requires constructible_from<CB, Initializer>
    explicit in_place_stop_callback(in_place_stop_token st, Initializer&&
        init)
        noexcept(is_nothrow_constructible_v<CB, Initializer>);

    ~in_place_stop_callback();

    // Not movable/copyable
    in_place_stop_callback(const in_place_stop_callback&) = delete;
    in_place_stop_callback(in_place_stop_callback&&) = delete;
    in_place_stop_callback& operator=(in_place_stop_callback&&) = delete;
    in_place_stop_callback& operator=(in_place_stop_callback&&) = delete;

private:
    in_place_stop_source* src; // exposition-only
    CB callback;
};

```

The semantics with regards to callback registration, deregistration and invocation of callbacks with respect to calls to `in_place_stop_source::request_stop()` is identical

to that of `stop_source` and `stop_callback`.

However there are some additional semantic constraints placed on the usage of the proposed in-place versions of the stop token types compared to the existing types:

- Any `in_place_stop_token` obtained from a call to `in_place_stop_source::get_token()` is associated with that `in_place_stop_source`. Any `in_place_stop_token` copied from an `in_place_stop_token` associated with an `in_place_stop_source` is also associated with that `in_place_stop_source`.
- Any `in_place_stop_token` associated with an `in_place_stop_source` is invalidated by the beginning of the invocation of the destructor of the `in_place_stop_source`. The only valid operations on an invalidated `in_place_stop_token` are to call the destructor or to assign a new `in_place_stop_token` value to the token. [[Note: this means that any calls to `stop_requested()` or `stop_possible()` must strongly happen-before the beginning of the invocation of the destructor of the associated `in_place_stop_source` object.]].
- Any `in_place_stop_callback` constructed using an `in_place_stop_token` associated with an `in_place_stop_source` is also associated with that `in_place_stop_source`.
- It is undefined behaviour if the program does not ensure that the return from the call to the destructor of all `in_place_stop_callback` objects associated with an `in_place_stop_source` object strongly happens before the beginning of the invocation of the destructor of the associated `in_place_stop_source` object.

§ 4.5 Add `get_stop_token()` customisation-point

Add the following definition to the `<execution>` header:

```
// <execution>
namespace std::execution
{
    inline namespace unspecified {
        inline constexpr unspecified get_stop_token = unspecified;
    }
}
```

Where `execution::get_stop_token` defines a customisation-point object that is invocable with a single argument that is an lvalue referencing an object whose type satisfies the `execution::receiver` concept such that `execution::get_stop_token(r)` is equivalent to:

- `std::tag_invoke(std::tag_t<get_stop_token>, std::as_const(r))` if this expression is valid;
- otherwise is equivalent to `std::never_stop_token{}`

The program is ill-formed if the decay-copied result of the expression `get_stop_token(r)` has a type that does not model the `std::stoppable_token` concept.

The program is ill-formed if customisations of the `get_stop_token()` customisation-point are not declared `noexcept`.

i.e. `noexcept(execution::get_stop_token(declval<decltype((r))>()))` must be `true`.

Note: See [P1895R0](#) “tag_invoke: A general pattern for supporting customisable functions” for details of the proposed `std::tag_invoke()`

Semantic Constraints

There are some additional semantic constraints applied to customisations of the `get_stop_token()` customisation-point and also to usage of this customisation point.

The `get_stop_token()` customisation-point is intended for use by implementations of `execution::connect()` for the sender to query the stop-token to use for receiving notification of stop-requests by calling `get_stop_token()` on the receiver passed as the second argument to `connect()`.

Note that the receiver represents the calling context. Other contextual information may also be passed to the operation implicitly via queries on the receiver. See the “Design Discussion” section on generalising context propagation.

The stop-token returned by `get_stop_token(receiver)` may only be assumed to be valid until the operation-state object returned by the call to `execution::connect()` is destroyed.

Conversely, customisations of `get_stop_token()` for a given receiver must ensure that the stop-token returned is valid until at least after the operation-state constructed when it was connected to a sender is destroyed.

Note that the operation-state object is allowed to be destroyed either before `execution::start()` is called or, if `execution::start()` is called, then after the beginning of a successful invocation of the completion-signalling operation on the receiver (i.e. `set_value`, `set_error` or `set_done`).

A receiver’s completion-signal handler will often go on to execute logic that ends up destroying the parent operation-state. And for many algorithms that introduce new cancellation-scopes, they will often be implemented by storing an `in_place_stop_source` in the parent operation-state and will customise the receiver passed to child operations so that its `get_stop_token()` will return an `in_place_stop_token` associated with this `in_place_stop_source`.

So when writing generic code that supports responding to stop-requests we have to assume that when we call `set_value()`, `set_error()` or `set_done()` on the receiver that this may

end up destroying the shared stop-state and thus invalidate any stop-tokens obtained from the receiver's `get_stop_token()` implementation.

This means that an operation that obtains stop-tokens from a receiver, `r`, by calling `get_stop_token(r)`, will need to ensure that:

- any use of stop-tokens obtained by calling `get_stop_token(r)` happens-before the return from the operation-state destructor; and
- the calls to destructors of any stop-callbacks constructed using stop-tokens obtained by calling `get_stop_token(r)` happen before the return from the operation-state destructor.

In many cases, the safest way to do this is to defer calling `get_stop_token()` and construction stop-callback objects until `execution::start()` is called and to ensure that stop-callback objects are destroyed before calling one of the completion-signalling operations (`set_value`, `set_error` or `set_done`).

§ 4.6 Type-traits

Add the following helper-trait to the `<execution>` header.

```
namespace std::execution
{
    template<typename Source>
    using stop_token_type_t =
        std::remove_cvref_t<decltype(execution::get_stop_token(std::declval<Source>
            ()))>;

    template<typename Source>
    struct stop_token_type
    {
        using type = stop_token_type_t<Source>;
    };
}
```

This trait is typically used to simplify the implementation of `operation_state` types returned from `execution::connect()`.

For example: An operation-state object may choose to declare a member that stores a stop-callback that is used to subscribe to the receiver's stop-token.

```
template<typename Receiver>
struct schedule_operation_state {
    struct cancel_callback {
        schedule_operation_state& op;
        void operator()() noexcept {
            op.context.cancel(&op);
        }
    };

    void start() noexcept {
```

```

    stopCallback.construct(execution::get_stop_token(receiver),
        cancel_callback{*this});
    context.enqueue(this);
}

thread_context& context;
Receiver receiver;
manual_lifetime<typename std::execution::stop_token_type_t<Receiver&>
    ::template callback_type<cancel_callback>> stopCallback;
};

```

It can also be used in conjunction with the `unstoppable_token` concept to check statically whether the receiver’s stop-token can ever be cancelled.

```

void my_operation_state<Receiver>::start() noexcept {
    // Avoid instantiating receiver's `set_done()` if stop can never be requested.
    if constexpr (
        (!std::unstoppable_token<std::execution::stop_token_type_t<Receiver>>)
        {
            if (std::execution::get_stop_token(receiver).stop_requested()) {
                // Stop already requested.
                // Complete immediately with set_done().
                std::execution::set_done(std::move(receiver));
                return;
            }
        }
    )
    // ... else start the operation
}

```

§ 5 Design Discussion

§ 5.1 Naming

This paper proposes several new names, all centred around the idea of a “stop token” that represents the thing an operation can poll or subscribe to to tell whether or not there has been a request to stop the operation.

However, the obvious name, `std::stop_token`, has already been taken by the type added to the standard library in C++20 which means we cannot use this as the name for the generalised concept.

To distinguish the concept of a “stop token” from the concrete `std::stop_token` type this paper chose to use the term `std::stoppable_token` as something close but distinct from the existing name.

The `std::stoppable_token` name is not ideal, however, for a couple of reasons:

- Not all types that model this concept are able to produce a stop request. i.e. not all types that match `std::stoppable_token` are “stoppable,” such as the proposed

`std::never_stop_token`.

- Technically the token is not “stoppable” but rather, “able to produce a stop request.” If anything were “stoppable” it would be the operation itself, but that is not what this concept is describing.

However, the author was unable to find a more suitable name than `stoppable_token` for this concept. The guidance for naming of concepts listed in [P1851R0](#) “Guidelines For snake_case Concept Naming” does not seem to prescribe a standard way for resolving concept name conflicts when an existing concrete type has already taken the obvious abstraction name other than by “using creativity.”

The `std::unstoppable_token` name is also similarly not ideal as the name suggests that this concept would match a set of types mutually exclusive from the types matched by `std::stoppable_token`, but actually the `std::unstoppable_token` concept subsumes `std::stoppable_token`. i.e. all `std::unstoppable_token` types are also `std::stoppable_token` types.

Another possible name for `std::unstoppable_token` is `std::never_stoppable_token` which uses the “never” terminology consistent with `std::never_stop_token`.

Alternatively, this potential confusion between `std::stoppable_token` and `std::unstoppable_token` could be resolved by replacing `std::unstoppable_token` concept with a `std::is_stop_ever_possible_v<StopToken>` trait (see further discussion below).

The naming of the new concrete types that model `std::stoppable_token` are:

- `std::in_place_stop_token`
- `std::never_stop_token`

These follow the general pattern of `<some_adjective>_stop_token`.

Associated types (if any) also follow the pattern of `<some_adjective>_stop_source` and `<some_adjective>_stop_callback` to mirror the naming conventions of the existing `std::stop_source` and `std::stop_callback` types added in C++20.

§ 5.2 Why do we need a `std::stoppable_token` concept?

The `std::stop_token` type added in C++20 is a vocabulary type that can be passed to an operation and later used to communicate an asynchronous request to stop that operation.

The design of `std::stop_token` defined a shared-ownership model that allows the lifetimes of associated `std::stop_source` and `std::stop_token` objects to be independent of each other. This is necessary for some use-cases, including their use in the `std::jthread` type, which allows a `std::jthread` to detach from the thread resource and destroy its `std::stop_source` before the thread has finished using the `std::stop_token`.

This shared-ownership model implies some runtime overhead, however, typically requiring a heap-allocation and atomic-reference counting of the shared state.

To avoid forcing this overhead on all async operations we want to allow other implementations to use other stop-token types that make different performance tradeoffs.

For example, allowing more efficient implementations, such as `std::in_place_stop_token`, for cases where usage is more structured - as sender/receiver usage is.

Or allowing no-op implementations, such as `std::never_stop_token`, for cases where cancellation is not required.

Thus, in cases where we want to write algorithms that work generically over different stop-token types it would be beneficial to allow parameters accepting a stop-token to be constrained with a concept that checks that the argument fulfills the syntactic requirements of the concept.

Finally, having a concept for this gives us somewhere to describe the semantic constraints on implementations of `std::stoppable_token` types.

§ 5.3 Should `unstoppable_token` concept just be a trait?

This paper has proposed the addition of a concept named `std::unstoppable_token` that refines the proposed `std::stoppable_token` concept to match only those stop-token types that statically guarantee they will never issue a stop-request.

This concept can be used in `if constexpr` predicates to avoid instantiating code-paths that would only be necessary in responding to stop-requests.

For example:

```
template<typename Receiver>
void my_operation_state<Receiver>::start() & noexcept {
    // Avoid instantiating Receiver's set_done() if unstoppable
    if constexpr
        (!std::unstoppable_token<std::execution::stop_token_type_t<Receiver>>)
    {
        if (std::execution::get_stop_token(this->receiver).stop_requested())
        {
            std::execution::set_done(std::move(this->receiver));
            return;
        }
    }

    // ... rest of start() implementation
}
```

It can also be used to constrain specialisations of an operation-state type to give a more efficient implementation if cancellation will never be requested.

For example:

```
template<typename Receiver>
class my_operation_state {
    // Default implementation...
};

template<typename Receiver>
requires
    std::unstoppable_token<std::execution::stop_token_type_t<Receiver>>
class my_operation_state {
    // Implementation optimised for no-cancellation...
};
```

However it's unclear whether or not this needs to be a concept or whether it could just be a predicate type-trait.

For example, defining a `std::is_stop_ever_possible_v<T>` trait equivalent to:

```
namespace std
{
    template<stoppable_token T>
    inline constexpr bool is_stop_ever_possible_v = !unstoppable_token<T>;
}
```

Naming this as the positive “is stop ever possible” would help avoid a double negative for the `if constexpr` use-cases where the code-path is only taken if a stop-request is possible, but would then require adding a negation to the `requires`-clause for class specialisations for the no-cancellation case.

§ 5.4 Can `stoppable_token_for` concept be recast as semantic requirements?

This paper proposes adding the multi-type concept, `stoppable_token_for`, which refines the `stoppable_token` concept by checking that we can construct a stop-callback for a specific given stop-token type, callback-type and callback-initializer type.

This concept can be used to constrain customisations of the `execution::connect()` method for particular senders to require that the stop-token obtained from the receiver can have a stop-callback attached that takes a particular callback-type.

For example: Constraining the `execution::connect()` customisation for a sender

```
template<typename Operation>
struct cancel_callback {
    Operation* op;
    void operator()() noexcept { /* logic for handling a stop-request. */ }
};

template<typename Receiver>
struct operation_state {
    explicit operation_state(Receiver&& r)
```

```

: receiver(std::move(r))
, stopCallback_(std::execution::get_stop_token(receiver), this)
{}

void start() { /* logic for launching operation */ }

Receiver receiver;
typename std::execution::stop_token_type_t<Receiver>::
    template callback_type<cancel_callback<operation_state>>
        stopCallback_;
};

class my_sender {

    template<std::execution::receiver R>
    requires std::stoppable_token_for<
        std::execution::stop_token_type_t<R>,
        cancel_callback<operation_state<R>>,
        operation_state<R>*>
    friend operation_state<R>
        tag_invoke(std::tag_t<std::execution::connect>,
                    my_sender&& self,
                    R&& receiver) {
        return operation_state<R>{(R&&)receiver};
    }
};

```

However, it's unlikely that there would be value in constraining `connect()` implementations like this. All `std::stoppable_token` types should have a nested `callback_type<CB>` type alias that can be instantiated with any type `CB` for which `std::invocable<CB> && std::destructible<CB>` is true.

Unfortunately, this kind of “universal quantification” is not something that we can currently express in a concept definition. So if we do want to express these constraints we are left with having to define a multi-type concept and then check this concept only once we know all of the concrete types.

One alternative direction to explore for specification could be to consider adding a semantic requirement that for a type, `T`, to satisfy the `std::stoppable_token` concept that the exposition-only concept `stoppable_token_for<T, CB, Initializer>` must be satisfied for all hypothetical pairs of types `CB` and `Initializer` where `CB` meets the requirements for `std::invocable` and `std::constructible_from<Initializer>`.

§ 5.5 Why do we need the `::callback_type<CB>` type-alias on the stop-token type?

As generic code needs to be able to support arbitrary `std::stoppable_token` types, and each of these types can have a different stop-callback type, we need some way for generic code to obtain the associated stop-callback type for a given stop-token type.

The way that generic code obtains this stop-callback type is through the nested template type-alias `T::callback_type<CB>`.

§ 5.6 Why doesn't paper this propose adding a `std::never_stop_callback<CB>` type-name?

In cases where you know statically that you're using a `std::stop_token` you can explicitly name the `std::stop_callback` type-name directly to construct a stop-callback that subscribes to notification of stop-requests.

Similarly, if you know statically that you're using a `std::in_place_stop_token`, you can explicitly name the `std::in_place_stop_callback` type-name directly.

In cases where you need to operate generically on any `std::stoppable_token`, you will need to use the `ST::callback_type<CB>` type-alias to lookup the corresponding stop-callback type for a given `std::stoppable_token` type, `ST`.

In cases where you know statically that you have a `std::never_stop_token`, there is no point to constructing a hypothetical `std::never_stop_callback` since you know the callback will never be invoked.

§ 5.7 Should `get_stop_token()` be applicable to more than receivers?

As proposed in this paper, the `execution::get_stop_token()` customisation point is limited to being applied to objects that model the receiver concept.

The reason for this is so that we can apply the semantic constraints on the validity of the stop-token returned from `execution::get_stop_token(r)` with relation to lifetime of the operation-state returned by `execution::connect(s, r)`.

However, it's possible that we may also find uses for the `execution::get_stop_token()` CPO as a mechanism for obtaining a stop-token from other kinds of objects. For example, there may be use-cases where we want to be able to apply the `execution::get_stop_token()` customisation-point to a coroutine promise-type to obtain the current coroutine's stop-token.

Thus we should consider specifying `get_stop_token()` to allow it to be called on other kinds of objects but done in such a way that the receiver-related semantic requirements are enforced when applied to a receiver passed to `execution::connect()`.

§ 5.8 Composability

One of the key design goals of this proposal is to allow generic composition of cancellable async operations. This section discusses some of the considerations around supporting this.

§ 5.8.1 Algorithms should be aware of or transparent to cancellation

For cancellation to be effective in an application that composes async operations using senders, we need to be able to issue a stop-request to a high-level operation and have that request propagated through to the leaf-operations. However, for this to be possible, every intervening algorithm that composes the senders needs to be forwarding the stop-request on to its child operations.

For simpler algorithms that do not introduce new cancellation scopes (ie. that do not generate their own stop-requests) they simply need to be transparent to cancellation.

The easiest way for algorithms to do this is to pass a receiver into child operations that forwards calls to `get_stop_token()` to the parent operation's receiver.

For example: The `transform()` algorithm is transparent to cancellation.

```
template<typename Src, typename Func>
class transform_sender {
    Src source;
    Func func;

    template<typename Receiver>
    struct operation_state {
        struct receiver {
            operation_state* state;

            template<typename... Values>
            void set_value(Values&&... values) && {
                std::execution::set_value(std::move(state->receiver),
                                          std::invoke(state->func,
                                                      (Values&&)values...));
            }

            template<typename Error>
            void set_error(Error&& error) && noexcept {
                std::execution::set_error(std::move(state->receiver),
                                          (Error&&)error);
            }

            void set_done() && noexcept {
                std::execution::set_done(std::move(state->receiver));
            }

            // Forward get_stop_token() to the parent receiver
            friend auto tag_invoke(tag_t<execution::get_stop_token>, const
                receiver& self) noexcept
                -> std::invoke_result_t<execution::get_stop_token, const
                Receiver&> {
                return std::execution::get_stop_token(self.state->receiver);
            }
        };

    };
};

operation_state(Src&& source, Func&& func, Receiver&& r)
: receiver(std::move(r))
```

```

    , func(std::move(func))
    , innerState(std::execution::connect(std::move(source),
        receiver{this}))
    {}

    void start() noexcept {
        std::execution::start(innerState);
    }

    Receiver receiver;
    Func func;
    std::execution::connect_result_t<Src, receiver> innerState;
};

template<typename Receiver>
operation_state<Receiver> connect(Receiver&& r) && {
    return operation_state<Receiver>{std::move(source), std::move(func),
        std::move(r)};
}
};

```

By forwarding the `get_stop_token()` CPO call to the parent receiver and returning the parent receiver's stop token, this means that if the transform-sender's child operation asks for the stop-token it will get the parent operation's stop-token and thus will observe any stop-requests send to the parent operation - stop requests transparently pass through the transform-operation.

Note that this forwarding of query/getter-style CPOs on the receiver can be further generalised to allow forwarding other kinds of queries on the receiver by adding a `tag_invoke()` overload that is generic over the CPO being forwarded.

For example, instead of writing the following overload for the receiver

```

// Forward get_stop_token() to the parent receiver
friend auto tag_invoke(tag_t<execution::get_stop_token>, const receiver&
    self) noexcept
-> std::invoke_result_t<execution::get_stop_token, const Receiver&> {
    return std::execution::get_stop_token(self.state->receiver);
}

```

we can write:

```

template<typename CPO>
requires std::invocable<CPO, const Receiver&>
friend auto tag_invoke(CPO cpo, const receiver& self)
noexcept(std::is_nothrow_invocable_v<CPO, const Receiver&>)
-> std::invoke_result_t<CPO, const Receiver&> {
    return static_cast<CPO&&>(cpo)(self.state->receiver);
}

```

This will still succeed in forwarding calls to `get_stop_token()` on the receiver but will now also support forwarding calls to other query-like CPOs. e.g. `get_scheduler(r)` to

get the current scheduler, or `get_allocator(r)` to get the current allocator, or `get_priority(r)` to get the priority of a particular operation.

Thus if we ensure that sender-based algorithms added to the standard library are specified in such a way that receivers that pass to child operations will forward receiver-query-like CPO-calls to the parent

Generalising this forwarding mechanism for receiver queries should be explored further but is a topic for another paper.

§ 5.8.2 Introducing new cancellation-scopes in a sender algorithm

Not every algorithm is going to be transparent to cancellation. Algorithms that introduce concurrency will often also introduce a new cancellation scope.

A cancellation scope allows cancellation of child operations independently of cancellation of the operation as a whole, while usually still allowing cancellation of the parent operation to propagate to cancellation of child operations.

For example, consider the `stop_when()` algorithm. It accepts two input senders; a source and a trigger, such that:

- If source completes before trigger it will cancel the trigger operation
- If trigger completes before source then it cancels the source operation.
- If the composed operation is cancelled then both source and trigger are cancelled.
- Once both source and trigger complete then the composed operation completes with the result from source.

In this instance, the `stop_when()` algorithm introduces a new cancellation scope so that it can independently request cancellation of the child operations.

A possible implementation strategy for such a `stop_when()` algorithm would be to do the following:

- Have the `stop_when` operation-state hold:
 - The receiver connected to the `stop_when` sender
 - A `std::in_place_stop_source`
 - A stop-callback that subscribes to the parent receiver's stop-token and that calls `.request_stop()` on the `in_place_stop_source` when a stop-request is made on the parent receiver's stop-token.
 - Operation-states for each of the source and trigger operations, connected to receivers generated internally by the `stop_when` algorithm.
 - A `std::atomic<uint8_t>` that is decremented when either of the operations completes and is used to determine when both operations have completed.

- Some storage space for holding the result of the source sender while waiting for the trigger sender to complete (if the source sender completes first)
- The receivers passed to `connect()` on both the source and trigger operations customise `get_stop_token()` to return a `std::in_place_stop_token` obtained from the `stop_when` operation's `std::in_place_stop_source`.

This pattern of an operation-state owning a stop-source, subscribing to the parent operation's stop-token to forward the stop-request onto the stop-source, and then passing a stop-token referencing the stop-source on to child operations by customising `get_stop_token()` on the receivers passed to those operations is a common pattern when implementing concurrency patterns that introduce a new cancellation scope.

See Appendix A for details of the implementation of `stop_when()`.

§ 5.8.3 Inhibiting cancellation propagation

There are sometimes cases where we don't want a stop-request issued to the parent operation to propagate to a child operation. For example, the child operation might be a cleanup operation which we want to run to completion regardless of whether the parent operation is cancelled or not.

This can be achieved by building a sender that wraps the connected receiver in a new receiver type that customises `get_stop_token()` to return `std::never_stop_token`.

For example:

```
template<typename Sender>
struct unstoppable_sender {
    Sender inner;

    template<template<typename...> class Variant,
            template<typename...> class Tuple>
    using value_types = typename Sender::template value_types<Variant,
        Tuple>;

    template<template<typename...> class Variant>
    using error_types = typename Sender::template error_types<Variant>;

    static constexpr bool sends_done = Sender::sends_done;

    template<typename Receiver>
    struct receiver {
        Receiver inner;

        // Override get_stop_token()
        friend std::never_stop_token tag_invoke(
            std::tag_t<std::execution::get_stop_token>,
            const receiver& self) noexcept {
            return {};
        }
    }
}
```

```

// Pass through other CPOs
template<typename CPO, typename Self, typename... Args>
requires (!std::same_as<CPO,
    std::tag_t<std::execution::get_stop_token>>) &&
    std::same_as<std::remove_cvref_t<Self>, receiver> &&
    std::invocable<CPO, member_t<Self, Receiver>, Args...>
friend auto tag_invoke(CPO cpo, Self&& self, Args&&... args)
noexcept(std::is_nothrow_invocable_v<CPO, member_t<Self,
    Receiver>, Args...>)
-> std::invoke_result_t<CPO, member_t<Self, Receiver>, Args...> {
return std::move(cpo)(static_cast<Self&&>(self).inner,
    static_cast<Args&&>(args)...);
}
};

template<typename Self, typename Receiver>
requires std::same_as<std::remove_cvref_t<Self>, unstoppable_sender>
    &&
    std::execution::receiver<Receiver> &&
    std::constructible_from<std::remove_cvref_t<Receiver>,
    Receiver> &&
    std::sender_to<member_t<Self, Sender>,
    receiver<std::remove_cvref_t<Receiver>>>
friend auto tag_invoke(std::tag_t<std::execution::connect>, Self&& self,
    Receiver&& receiver)
noexcept(
    std::is_nothrow_constructible_v<std::remove_cvref_t<Receiver>,
    Receiver> &&
    std::is_nothrow_invocable_v<decltype(std::execution::connect),
        member_t<Self, Sender>,
        receiver<std::remove_cvref_t<Receiver>>>
-> std::invoke_result_t<decltype(std::execution::connect),
    member_t<Self, Sender>,
    receiver<std::remove_cvref_t<Receiver>>> {
// Wrap the incoming receiver and forward through to Sender's
// connect().
return std::execution::connect(
    static_cast<Self&&>(self).inner,
    receiver<std::remove_cvref_t<Receiver>>>{static_cast<Receiver&&>
        (receiver)});
}
};

```

Then you can wrap your operation in the `unstoppable_sender` and stop-requests from the parent will no longer propagate to the child operation.

§ 5.8.4 Coroutine integration / limitations of `std::task`

The design of the `std::task`/lazy coroutine type proposed in [P1056R1](#) does not support the design goal of generically composable, cancellable operations.

This [P1056R1](#) design was largely modelled on the design of `cppcoro::task` and the limitations of this with regards to cancellation have been discussed in prior sections.

The implementation and usage experience of `folly::coro::Task` in Facebook has shown that the model of implicit propagation of a `CancellationToken` can be used to provide a simple interface for ensuring that cancellation of a high-level operation implicitly propagates that request to child operations.

We have implemented a prototype of a `task<T>` coroutine-type in [libunifex](#) that supports the same ideas for implicit propagation of cancellation-context from parent coroutine to child coroutine as `folly::coro::Task` but with the implementation revised to integrate with the sender/receiver concepts and the cancellation-mechanism proposed in this paper.

This implementation needs to support 3 cases for propagating stop-requests through a task coroutine: 1. Where a task is used as a child of a sender-based algorithm. The receiver passed to the task's `connect()` operation needs to have its stop-token's stop-requests forwarded into child operations of the task. 2. Where a task coroutine awaits a sender we need to make sure that the task injects a receiver into the call to `connect()` on the awaited sender that has customised `get_stop_token()` to return the task's stop-token inherited from its parent. 3. Where a task awaits an awaitable type, such as another task, the task's stop-token needs to be propagated to that awaitable while still preserving the ability to symmetrically-transfer execution to a child coroutine - something that is not possible if indirecting through a sender's `connect/start` interface.

For example:

```
static_thread_pool tp;

task<int> child1(auto scheduler) {
    co_await schedule_after(tp, 1s);
}

task<int> child2(auto scheduler) {
    //
    co_await schedule_after(tp, 10ms);
    throw std::runtime_error{"failed"};
}

task<int> parent(auto scheduler) {
    // Passing tasks into when_all() which treats the task as a sender.
    // when_all() will inject its own stop-token into the child tasks.
    //
    // When child2() completes with an error after 10ms this should
    // cancel child1() quickly rather than waiting for the full 1s.
    //
    // Awaiting sender-result of when_all() - this needs to have the
    // stop-token from the parent() task injected into the sender so
    // that cancelling parent() cancels the when_all() operation.
    co_await when_all(child1(scheduler), child2(scheduler));
}
```

The `libunifex` prototype also explores some strategies for representing a 'done' result from a sender when awaited within a task-coroutine similar to an exception unwind but one that is not catchable with a `try/catch`.

Even though the ‘done’ signal cannot be caught with a try/catch, we can still apply sender-algorithms to translate the ‘done’ signal into either the value-channel, e.g. by returning a `std::optional`, or into the error-channel, e.g. by throwing an `operation_cancelled` exception.

For example:

```
sender_of<int> auto some_cancellable_operation();

task<void> example() {
    // If this completes with 'set_done' then this will unwind the
    // awaiting coroutine and it will also complete with the 'done'
    // signal.
    int x = co_await some_cancellable_operation();

    // But we can apply an algorithm that translates the 'done'
    // signal into a value.
    std::optional<int> y = co_await
        done_as_optional(some_cancellable_operation());
    if (!y.has_value()) {
        // Completed with cancellation.
    }

    // Or we can translate the 'done' signal into an error.
    try {
        int z = co_await done_as_error<operation_cancelled>
            (some_cancellable_operation());
    } catch (const operation_cancelled&) {
        // Handle cancellation
    }
}
```

Note that options for treating the ‘done’ signal as first-class within a coroutine and in non-coroutine functions have been explored in [P1677R2](#) - “Cancellation is Serendipitous Success” (by Kirk Shoop and Lisa Lippincott).

There is still some further design work required to investigate and incorporate other capabilities into a revised task design before a revision to P1056 can be produced, including:

- generalising the context-propagation support within task to allow propagating other kinds of context from caller to callee. e.g. an allocator, a scheduler, or some application-specific context
- supporting a task having an associated scheduler and always resuming on the execution context of that task
- allocator customisation - this is also explored by the paper [P1681R0](#)
 - “Revisiting allocator model for coroutine lazy/task/generator” (by Gor Nishanov)

Support for propagating cancellation signals through coroutines is not part of this proposal. However, cancellation support *should* be incorporated into a subsequent revision of [P1056](#). The author does not believe that [P1056R1](#) should be accepted as-is due to its poor support for composable cancellation.

§ 5.9 Cancellation is optional / best-effort

The intention is to allow cancellation to be opt-in for both the implementation of a sender-based async operation and for the consumer of that operation (i.e. the author of the receiver).

If a receiver, `r`, does not customise the `get_stop_token()` customisation-point to return a stop-token that would allow it to communicate a stop-request then when the sender calls `get_stop_token(r)` on the receiver it will dispatch to the default version, which returns `std::never_stop_token`.

If the sender tries to use this stop-token to respond to a stop-request the compiler will see an empty type with both `stop_possible()` and `stop_requested()` statically returning `false`. This should allow the compiler to optimise out most code-paths that would normally be dealing with stop-requests.

Conversely, if a sender does not support cancellation it does not need to call `get_stop_token()` and does not need to respond to stop-requests. In this case there is no overhead or extra complexity required in the sender implementation to ignore stop-requests, it can simply just ignore the `get_stop_token()` customisation-point altogether and let the async operation naturally run to completion.

Note that, in general, responding to a request to stop is inherently racy as the source of the request to stop is potentially executing concurrently with the natural completion of the operation (cancellation almost always involves some form of concurrency). So it's always possible that a request to stop comes too late and is ignored because the operation has progressed past the point where it can be cancelled.

Thus async operations often only respond to cancellation on a best-effort basis.

For example, at the time that you request cancellation of an async I/O operation for which you have not yet received notification of its completion, the I/O may actually have already have completed and the OS has posted the completion notification and it's just sitting in a queue waiting for you to process it. In this situation, the OS will almost certainly just ignore the request to cancel an already-complete I/O operation.

Stop-requests can also be ignored simply because the operation does not support cancellation.

Thus applications will generally need to be able to cope with stop-requests that are ignored. The timeliness of responding to a stop-request, or whether it responds at all to a stop-request, can often be a QoI decision for the implementation of that operation.

However, there are some operations that may require support for cancellation to be able to build a correct application. For example, a server that listens for incoming connections on a socket may need to be able to cancel the `accept()` operation during shutdown to handle the case where no more clients will attempt to establish connections. If the `accept()` operation did not respond to a stop-request then the program may never terminate.

§ 5.10 Performance Considerations

This section discusses several of the performance considerations that went into the design of this proposal.

§ 5.10.1 Don't pay for what you don't use

Supporting cancellation of an async operation generally has runtime overhead compared to operations that do not support cancellation. Extra synchronisation, extra branches and extra storage for stop-callbacks is often required when supporting cancellation.

If we know at compile-time that a caller will never request cancellation of an operation then we'd like to be able to avoid the runtime overhead that comes with supporting cancellation.

The default implementation of the `get_stop_token()` customisation-point returns a `std::never_stop_token` which has `constexpr stop_possible()` and `stop_requested()` methods and also has an empty, no-op stop-callback type.

Consumers that do not opt-in to the ability to submit a stop-request by customising the `get_stop_token()` customisation-point will therefore end up providing a `std::never_stop_token` to the operation. Even if the operation does support cancellation, attempts to use this token type will compile out most cancellation-handling code-paths as dead-code and thus eliminate runtime cancellation overhead.

For cases where a fundamentally different and more efficient implementation is possible when cancellation is not required to be supported, the implementation can specialise on or use `if constexpr` in conjunction with the `std::unstoppable_token` concept to dispatch to the different implementations.

For example, the `libunifex::win32::windows_thread_pool` schedule operation adds two overloads of `connect()`, one for receivers whose stop-token is never going to produce a stop-request and which returns an operation-state that takes a more efficient approach, and another overload for receivers that might issue a stop-request.

```
template<typename Receiver>
class schedule_op {
    // non-cancellable version ...
};

template<typename Receiver>
class cancellable_schedule_op {
```

```

// cancellable version ...
};

class windows_thread_pool::schedule_sender {
public:
    // Dispatch to cancellable implementation if unstoppable
    template<typename Receiver>
        requires std::execution::receiver_of<Receiver> &&

        std::unstoppable_token<std::execution::stop_token_type_t<Receiver>>)
    schedule_op<unifex::remove_cvref_t<Receiver>> connect(Receiver&& r)
        const {
        return schedule_op<unifex::remove_cvref_t<Receiver>>{
            *pool_, (Receiver&&)r};
        }

    // Dispatch to cancellable implementation if not unstoppable
    template<typename Receiver>
        requires receiver_of<Receiver> &&

        (!std::unstoppable_token<std::execution::stop_token_type_t<Receiver>>)
    cancellable_schedule_op<unifex::remove_cvref_t<Receiver>>
        connect(Receiver&& r) const {
        return cancellable_schedule_op<unifex::remove_cvref_t<Receiver>>{
            *pool_, (Receiver&&)r};
        }

    ...
};

```

Implementations can also avoid expensive operations needed to support cancellation at runtime, even if this is not known statically, by calling the `.stop_possible()` member function on the stop-token.

For example, the `cancellable_schedule_op` type mentioned above requires a heap allocation to support cancellation, but only conditionally allocates this memory if `get_stop_token(receiver).stop_possible()` is true.

§ 5.10.1.1 Trying to cancel uncancellable operations

The one case where it is more difficult to eliminate all runtime overhead is where a consumer of an operation that introduces a new cancellation scope might request cancellation of that operation but where the operation does not support cancellation and never calls `get_stop_token()` on the receiver.

As there is no query available to ask a sender whether or not it will respond to a stop-request the consumer will have to assume it might and reserve storage for a stop-source, e.g. a `std::in_place_stop_source` which could be 16 bytes.

In the case where a stop-request is made there will still usually involve at least one atomic operation to signal the stop-request, although if `get_stop_token()` was never called it would never need to execute any stop-callbacks.

§ 5.10.2 Avoiding heap-allocations and reference counting

The design of `std::stop_token` uses a shared-ownership model where the ownership of the stop-state is shared between all `std::stop_token`, `std::stop_source` and `std::stop_callback` objects associated with that stop-state.

This design was necessary to support independence in the respective lifetimes of `stop_source`-owners and `stop_token`-owners which is required for some use cases. For example, the `detach()` method included in `std::jthread` allows destruction of the `std::jthread` (which owns a `stop_source`) before the thread completes.

This shared-ownership model generally requires implementations to heap-allocate and atomically reference-count this shared stop-state. The overhead of this can make `std::stop_token` unsuitable for some high-performance use-cases requiring support for cancellation.

Ideally, we'd like to be able to avoid the runtime overhead of both the heap-allocation and reference-counting, but doing so requires placing more restriction on the use of a stop-token abstraction.

For example, if we did not allow the stop-source object to be movable or copyable and we required that the lifetime of stop-token/stop-callback objects was nested within the lifetime of a single, associated stop-source object then this would allow storing the stop-state inline inside the stop-source object, avoiding the need for a heap-allocation. It would also eliminate the need for reference counting since we know, by construction, that the stop-source will always be the last reference to the shared stop-state.

It just so happens that sender-based algorithms that provide the structured concurrency guarantee have a usage model that exactly matches this more restrictive interface requirements. i.e. that child operations (users of stop-tokens/stop-callbacks) are required to complete before the parent operation (owner of the stop-source) completes. The stop-source can also be constructed in-place in the parent operation's operation-state object. As the operation-state object itself is not movable/copyable, the stop-source object does not need to be movable/copyable.

The `std::in_place_stop_source` type and its associated `std::in_place_stop_token` and `std::in_place_stop_callback` types proposed in this paper provide an implementation of the `std::stoppable_token` concept that has this more restrictive usage model compatible with usage in sender-based algorithms that adhere to the structured concurrency guarantee.

This allows sender algorithms introducing new cancellation scopes to use a stop-token based cancellation-model without the need for heap-allocations or atomic reference counting. This should allow more efficient implementations compared to what is possible with the more general `std::stop_token` interface.

It is worth noting, however, that implementations are still free to use `std::stop_token` if desired, as it is also a valid implementation of the `std::stoppable_token` concept.

§ 5.10.3 Type-erasure of stop callbacks

The stop-token design supports registering multiple stop-callbacks to receive notification of stop requests made from a given stop-source. Implementations of the stop-token concept will therefore generally need to maintain a list of registered callbacks and as each callback can potentially have a different type, this will require some form of type-erasure of the callbacks.

This type-erasure can be implemented without the need for heap-allocations, although it does mean that making a stop-request will involve making an indirect function-call to invoke each registered callback.

While the cost of this indirect call is not expected to be significant, it is worth noting that the cancellation model used by the Networking TS, which involves a parent operation directly calling a `.cancel()` method on a child object, does not have this same inherent need for type-erasure of the cancellation logic.

§ 5.10.4 Synchronisation required to support stop-requests coming from other threads

The existing `std::stop_token` family of types, as well as the proposed `std::in_place_stop_token` family of types, are both designed to allow stop-requests to be made from one thread while another thread is either polling for stop-requests or registering a stop-callback.

Supporting the ability to make a stop-request from any thread makes it easier to build cancellation algorithms as you don't have to worry about figuring out whether or not it's safe to issue a stop-request from the current thread and if not then figuring out which execution context is associated with the child operation you want to cancel and then scheduling work onto that execution context.

However, this capability means that the implementation of these types necessarily involves some form of thread synchronisation to ensure that this is safe - typically some atomic operations and a spin-lock held for a short period of time.

This design approach for cancellation is different to that of the Networking TS, which usually requires that calls to request cancellation are serialised with respect to calls to other operations on a given I/O object. Serialisation of these calls is usually handled by scheduling all work that might access the I/O object onto a strand-executor. There is still thread-synchronisation here, it's just been moved out of the I/O object/operation and into the strand executor.

Note that even with this model, care needs to be taken to correctly handle the case where a call to the `.cancel()` method is scheduled onto the strand-executor's queue and the

operation completes concurrently before the `.cancel()` call can be evaluated.

The design of stop-callbacks and the requirements placed on their implementations by the `std::stoppable_token` concept is intended to solve this problem in a different way by allowing you to synchronously deregister a callback in such a way that you are guaranteed that after the deregistration completes that there is no other thread that is or will concurrently execute that stop-callback.

§ 5.10.5 Allowing optimised implementations for single-threaded use-cases

For use-cases where an application is only ever running logic on a single thread and we know that all stop requests will be made on that thread and all usages of the stop-token will also occur on that thread then the overhead of the thread-synchronisation inherent in the `std::stop_token` and `std::in_place_stop_token` types is unnecessary and may wish to be avoided.

However, if a given algorithm that introduces a new cancellation-scope in this environment has been written in terms of `std::in_place_stop_token` then it becomes difficult to avoid its inherent synchronisation, even if it's only ever accessed from a single thread.

It's an open-question whether or not we need to support some kind of mechanism to allow applications that only perform single-threaded cancellation to avoid the thread-synchronisation overhead.

More design investigation is required to be able to determine how best to do this within the sender/receiver framework.

§ 5.10.6 Cost of `get_stop_token()` for deep operation stacks

The design proposed in this paper, where an operation obtains the stop-token to use by calling `get_stop_token()` on the receiver passed to `connect()`, will potentially have many algorithms that are transparent to cancellation. i.e. where it just forwards the `get_stop_token()` through to the parent receiver

Usually, the receiver passed to child operations of a transparent-to-cancellation algorithm will hold a pointer to the parent operation-state and the parent receiver will be held as a data-member of the parent operation-state.

This means that customising the `get_stop_token()` call on the child receiver to forward to the parent receiver will often involve a pointer indirection.

If many sender-operations have been composed into a deep hierarchy then this can mean that each call to `get_stop_token()` at the leaf-level may end up needing to walk $O(\text{depth})$ pointer indirections before we get to the receiver that is able to provide the concrete stop-token object.

If a particular high-level operation ends up having a large number of leaf operations, each of which call `get_stop_token()` then this chain of receivers may end up needing to be walked many times, which could be a performance bottleneck for some applications.

This can be overcome, however, by introducing special context-caching adapters in the call-stack that can cache these values and store them closer (in terms of number of pointer indirections) to the usage of that context.

Such an adapter would, upon initialisation in `connect()`, obtain the context from the parent receiver and then store a cached copy of that context in either the receiver or operation-state of that node.

For example: A `context_caching_sender` adapter that caches the value in a receiver adapter.

```
template<typename CPO, typename InnerSender>
struct context_caching_sender {
    InnerSender inner;

    template<typename InnerReceiver>
    struct receiver_wrapper {
        using value_type = std::invoke_result_t<CPO, const InnerReceiver&>;

        // Populates the cached value by invoking the CPO on the receiver.
        explicit receiver_wrapper(InnerReceiver r)
            : inner(std::move(r))
            , cached_value(CPO{}(inner))
        {}

        InnerReceiver inner;
        value_type cached_value;

        template<typename OtherCPO, typename Self, typename... Args>
        requires
            std::same_as<std::remove_cvref_t<Self>, receiver_wrapper> &&
            std::invocable<OtherCPO, member_t<Self, InnerReceiver>, Args...>
        friend auto tag_invoke(OtherCPO cpo, Self&& self, Args&&... args) {
            return cpo(static_cast<Self&&>(self).inner, static_cast<Args&&>
                (args)...);
        }

        // Hook that CPO to return the cached value instead of forwarding on
        // to the wrapped receiver.
        template<typename Self>
        friend const value_type& tag_invoke(CPO cpo, const receiver_wrapper&
            r) {
            return r.cached_value;
        }
    };

};

template<typename Self, typename Receiver>
requires std::same_as<std::remove_cvref_t<Self>,
    context_caching_sender>
```

```

friend auto tag_invoke(std::tag_t<std::execution::connect>, Self&& self,
    Receiver r) {
    return std::execution::connect(
        static_cast<Self&&>(self),
        receiver_wrapper<R>{std::move(r)});
}
};

```

Applying this adapter at key points within your application should allow you to address any O(depth)-related performance problems that arise.

§ 5.11 Limitations of `sender_traits::sends_done`

The `sender_traits` facility proposed in P0443R14 allows you to query what signals a give sender might complete with.

For example, the `sender_traits::error_types` type-alias lets you query what overloads of `set_error()` might be invoked on the receiver connected to it, and similarly, the `value_types` type-alias lets you query what overloads of `set_value()` might be invoked.

There is also the `sender_traits::sends_done` boolean static member that indicates whether or not the operation might complete with `set_done()`.

These queries can be used to pre-reserve storage for results, generate vttables, avoid template instantiations and apply various other optimisations. They are the equivalent to the `noexcept` and `decltype` expressions for regular functions, but in the async domain.

For many senders, whether or not the operation will complete with `set_done()` depends entirely on whether or not the receiver that it is connected to returns a `std::unstoppable_token` from its `get_stop_token()` customisation or not. If you never cancel the operation then it never completes with `set_done()`.

However, when querying information about the sender we do not yet know what receiver type it will be connected to and so the calculation of `sends_done` needs to assume that it *might* be connected to a receiver whose stop token is able to deliver a stop-request.

As senders need to conservatively report that they might complete with `set_done` this can lead to some missed optimisation opportunities for algorithms that might otherwise be able to

There are other similar limitations with respect to the `value_types` and `error_types` members of `sender_traits`.

The list of error-types that a sender produces may be dependent on the receiver in that whether or not it completes with a `std::exception_ptr` may be dependent on whether or not the `set_value()` overloads on the receiver that receive the result of the operation declared `noexcept` or not.

The value result-type of a sender might depend on some type-information obtained from the receiver it is connected to. For example, the sender might call a `get_allocator()` customisation point on the receiver to obtain an allocator of type, `A`, and then produce a result of type `std::vector<int, A>` constructed using that allocator.

One possible avenue for investigation here is to defer calculating the completion signals of a sender until we know what concrete receiver type is going to be connected to it. For example, we could replace `sender_traits<S>` with an `operation_traits<S, R>` type that allowed computing the result-type with full knowledge of the receiver type.

§ 5.12 Cancellation support for scheduler implementations

The executor concept proposed by P0443R14 and earlier revisions describe an interface for scheduling work onto a given execution context by calling the `std::execution::execute()` customisation point and passing the executor and an invocable object.

However, once a call to `execute()` is made, the work is enqueued and there is no standard way for the caller to be able to cancel this work to remove it from the queue if the caller later determines that this work no longer needs to be performed.

The scheduler concept proposed by P0443R14 also describes the ability to schedule work onto its associated execution context, but does so using the same sender/receiver concepts used for other async operations.

This means that the `schedule()` operation produced by a scheduler can make use of the same mechanisms used for other senders to support cancellation of the operation. i.e. by connecting the sender returned from `schedule()` to a receiver that has customised the `get_stop_token()` customisation-point.

§ 5.13 Impacts to other proposals

This paper has impacts on or is related to a number of other proposals.

This paper is primarily proposing an extension to the sender/receiver model proposed in P0443R14 - “A Unified Executors Proposal for C++” to add a standard, composable mechanism for cancellation of in-flight sender-based asynchronous operations.

While this paper could be applied as-is on top of P0443, the `get_stop_token()` customisation point is currently specified in terms of `tag_invoke()` facilities proposed in P1895R0 “tag_invoke: A general mechanism for supporting customisable functions” and so merging the two would also introduce a dependency on P1895.

The dependency on `tag_invoke()` should also be considered in conjunction with the papers P2221R0 “define P0443 cpos with tag_invoke” and P2220R0 “redefine properties

in P0443,” which also propose adoption of `tag_invoke()` facilities within the executors proposal.

Some of the code examples in this paper assume that changes proposed in P2221R0 have been applied.

This paper highlights some limitations with the current design proposed for the `std::lazy/std::task` type in P1056R1 with regards to the inability for these coroutine types to be able to be composed using generic algorithms in a way that allows them to participate in cancellation.

It is recommended that P1056 be updated to incorporate support for composable cancellation as proposed in this paper, although there are also other general changes required to support integration with sender/receiver.

The paper P1897 “Towards C++23 executors: A proposal for an initial set of algorithms” includes a number of sender-based algorithms. We should ensure that these algorithms are specified in a way that makes them either transparent to cancellation (e.g. `transform()`) or explicitly specify their cancellation behaviour if they introduce new cancellation scopes (e.g. `when_all()`) - In general, any algorithm that introduces concurrency should be evaluated for whether or not it should be introducing a new cancellation scope.

§ 5.14 Future work

There are also some areas for further investigation related to this paper.

- Investigate cancellability of `execution::execute()` operations.
- Extend `std::condition_variable_any::wait()` functions with support for arbitrary `std::stoppable_token` types rather than just `std::stop_token`.
- Add some basic cancellation-related algorithms:
 - `stop_when(src, trigger)` - issues a stop-request to `src` when `trigger` completes
 - `first_successful(inputs...)` - completes with first successful (ie. ‘value’) result, otherwise last result
 - `timeout(src, scheduler, duration)` -

§ 6 Implementation Experience

The general model of having a `CancellationToken` implicitly propagated to child coroutines has been implemented in `folly::coro`, Facebook’s library of coroutine abstractions, which has been used extensively in production.

The specific model for cancellation described in this paper, which integrates with the sender/receiver model for async computation proposed in P0443R14, has been

implemented in Facebook's libunifex opensource library.

A prototype implementation of a coroutine task-type that implicitly propagates a stop-token object to child coroutines and child senders has also been implemented as part of the libunifex library as `unifex::task<T>`. However, this coroutine type is not being proposed by this paper.

§ 7 Wording

Will be provided in a future revision of this paper.

§ 8 Appendices

§ 8.1 Appendix A: The `stop_when()` algorithm

This example shows how to implement libunifex's `stop_when()` algorithm which introduces a new cancellation-scope - where child operations can be cancelled independently of whether the parent operation was cancelled or not.

```
#include <stop_token>
#include <execution>
#include <atomic>
#include <optional>
#include <variant>
#include <tuple>
#include <type_traits>

namespace _stop_when {

struct forward_stop_request {
    std::in_place_stop_source& stopSource;

    void operator()() noexcept {
        stopSource.request_stop();
    }
};

template<typename Source, typename Trigger, typename Receiver>
struct _op;

template<typename Source, typename Trigger, typename Receiver>
struct _source_receiver {
    using op_t = _op<Source, Trigger, Receiver>;

    template<typename... Values>
    void set_value(Values&&... values) &&
        noexcept(std::is_nothrow_constructible_v<std::decay_t<Values>,
            Values> && ...) {
        op->result.template emplace<std::tuple<
            std::tag_t<std::execution::set_value>,
            std::decay_t<Values>...>>(>
```

```

        std::execution::set_value, (Values&&)values...);
    op->notify_child_complete();
}

template<typename Error>
void set_error(Error&& error) && noexcept {
    op->result.template emplace<std::tuple<
        std::tag_t<std::execution::set_error>, std::decay_t<Error>>>>(
        std::execution::set_error, (Error&&)error);
    op->notify_child_complete();
}

void set_done() && noexcept {
    op->result.template
        emplace<std::tuple<std::tag_t<std::execution::set_done>>>>(
        std::execution::set_done);
    op->notify_child_complete();
}

    op_t* op;
};

template<typename Source, typename Trigger, typename Receiver>
struct _trigger_receiver {
    using op_t = _op<Source, Trigger, Receiver>;

    template<typename... Values>
    void set_value(Values&&...) && noexcept {
        op->notify_child_complete();
    }

    template<typename Error>
    void set_error(Error&&) && noexcept {
        op->notify_child_complete();
    }

    void set_done() && noexcept {
        op->notify_child_complete();
    }

    friend std::in_place_stop_token tag_invoke(
        std::tag_t<std::execution::get_stop_token>,
        const _trigger_receiver& self) noexcept {
        return self.op->stopSource.get_token();
    }

    op_t* op;
};

template<typename... Values>
using value_result_tuple_t =
    std::tuple<std::tag_t<std::execution::set_value>,
        std::decay_t<Values>...>;

template<typename Source, typename Trigger, typename Receiver>
struct _op {

```

```

using source_receiver_t = _source_receiver<Source, Trigger, Receiver>;
using trigger_receiver_t = _trigger_receiver<Source, Trigger, Receiver>;

template<typename Receiver2>
explicit _op(Source&& source, Trigger&& trigger, Receiver2&& receiver)
: receiver(static_cast<Receiver2&&>(receiver))
, sourceOp(std::execution::connect((Source&&)source,
    source_receiver_t{this}))
, triggerOp(std::execution::connect((Trigger&&)trigger,
    trigger_receiver_t{this}))
{}

void start() && noexcept {
    // Subscribe to stop-requests from the parent.
    stopCallback.emplace(
        std::execution::get_stop_token(receiver),
        forward_stop_request{stopSource});

    // And start child operations.
    std::execution::start(triggerOp);
    std::execution::start(sourceOp);
}

void notify_child_complete() noexcept {
    stopSource.request_stop();
    if (remaining.fetch_sub(1, std::memory_order_acq_rel) == 1) {
        stopCallback.reset();
        deliver_result();
    }
}

void deliver_result() noexcept {
    try {
        std::visit([&](auto&& resultTuple) {
            constexpr size_t tupleSize =

                std::tuple_size_v<std::remove_reference_t<decltype(resultTuple)>>;
            if constexpr (tupleSize > 0) {
                std::apply(resultTuple, [&](auto completionFn, auto&&... args) {
                    completionFn(std::move(receiver), static_cast<decltype(args)>
                        (args)...);
                });
            } else {
                // Result not initialised (should be unreachable)
                std::terminate();
            }
        }, std::move(result));
    } catch (...) {
        std::execution::set_error(std::move(receiver),
            std::current_exception());
    }
}

template<typename... Errors>
struct error_result {
    template<typename... ValueTuples>

```

```

using apply = std::variant<
    std::tuple<>,
    std::tuple<std::tag_t<std::execution::set_done>>,
    ValueTuples...,
    std::tuple<std::tag_t<std::execution::set_error>,
        std::decay_t<Errors>>...>;
};

using source_traits_t = std::execution::sender_traits<Source>;

using result_t =
    typename source_traits_t::template value_types<
        value_result_tuple_t,
        typename source_traits_t::template
            error_types<error_result>::template apply>;

using parent_stop_token = std::execution::stop_token_type_t<Receiver>;
using stop_callback =
    typename parent_stop_token::template
        callback_type<forward_stop_request>;

Receiver receiver;
std::in_place_stop_source stopSource;
std::atomic<std::uint8_t> remaining{2};
std::optional<stop_callback> stopCallback;
result_t result;
std::execution::connect_result_t<
    Source, _source_receiver<Source, Trigger, Receiver>> sourceOp;
std::execution::connect_result_t<
    Trigger, _trigger_receiver<Source, Trigger, Receiver>> triggerOp;
};

template<typename Source, typename Trigger>
struct _sender {
    template<template<typename...> class Tuple,
        template<typename...> class Variant>
    using value_types = typename Source::template value_types<Tuple,
        Variant>;

    template<template<typename...> class Variant>
    using error_types = typename Source::template error_types<Variant>;

    static constexpr bool sends_done = Source::sends_done;

    template<typename Self, typename Receiver>
        requires std::same_as<std::remove_cvref_t<Self>, _sender> &&
            std::execution::receiver<Receiver> &&

    friend auto tag_invoke(
        std::tag_t<std::execution::connect>,
        Self&& self,
        Receiver&& receiver)
        -> _op<member_t<Self, Source>, member_t<Self, Trigger>,
            std::remove_cvref_t<Receiver>> {

```



```

        return _op<member_t<Self, Source>, member_t<Self, Trigger>,
            std::remove_cvref_t<Receiver>>{
            static_cast<Self&&>(self).source,
            static_cast<Self&&>(self).trigger,
            static_cast<Receiver&&>(receiver)
        };
    };

    Source source;
    Trigger trigger;
};

struct _fn {
    // Dispatch to custom implementation if one provided.
    template<typename Source, typename Trigger>
    requires
        std::execution::sender<Source> &&
        std::execution::sender<Trigger> &&
        std::tag_invocable<_fn, Source, Trigger>
    auto operator()(Source&& source, Trigger&& trigger) const
    noexcept(std::is_nothrow_tag_invocable_v<_fn, Source, Trigger>)
    -> std::tag_invoke_result_t<_fn, Source, Trigger> {
        return std::tag_invoke(_fn{}, (Source&&)source,
            (Trigger&&)trigger);
    }

    // Otherwise fall back to default implementation
    template<typename Source, typename Trigger>
    requires
        std::execution::sender<Source> &&
        std::execution::sender<Trigger> &&
        (!std::tag_invocable<_fn, Source, Trigger>) &&
        std::constructible_from<std::remove_cvref_t<Source>, Source>
        &&
        std::constructible_from<std::remove_cvref_t<Trigger>, Trigger>
    auto operator()(Source&& source, Trigger&& trigger) const
    noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<Source>,
        Source> &&

        std::is_nothrow_constructible_v<std::remove_cvref_t<Trigger>,
            Trigger>)
    -> _sender<remove_cvref_t<Source>, remove_cvref_t<Trigger>> {
        return _sender<remove_cvref_t<Source>, remove_cvref_t<Trigger>>{
            (Source&&)source,
            (Trigger&&)trigger
        };
    }
};

} // namespace _stop_when

inline constexpr _stop_when::_fn stop_when;

```